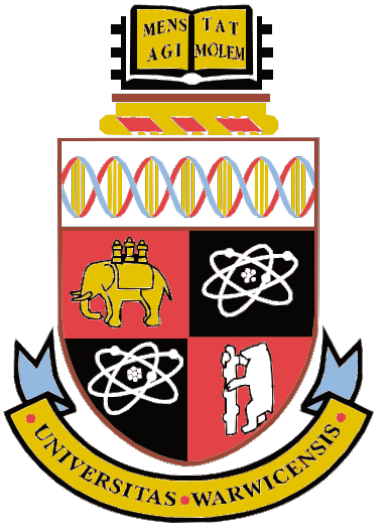


Real-time Simulations of Momentum-Conserving Vortices using Affine Velocity Approximations

by

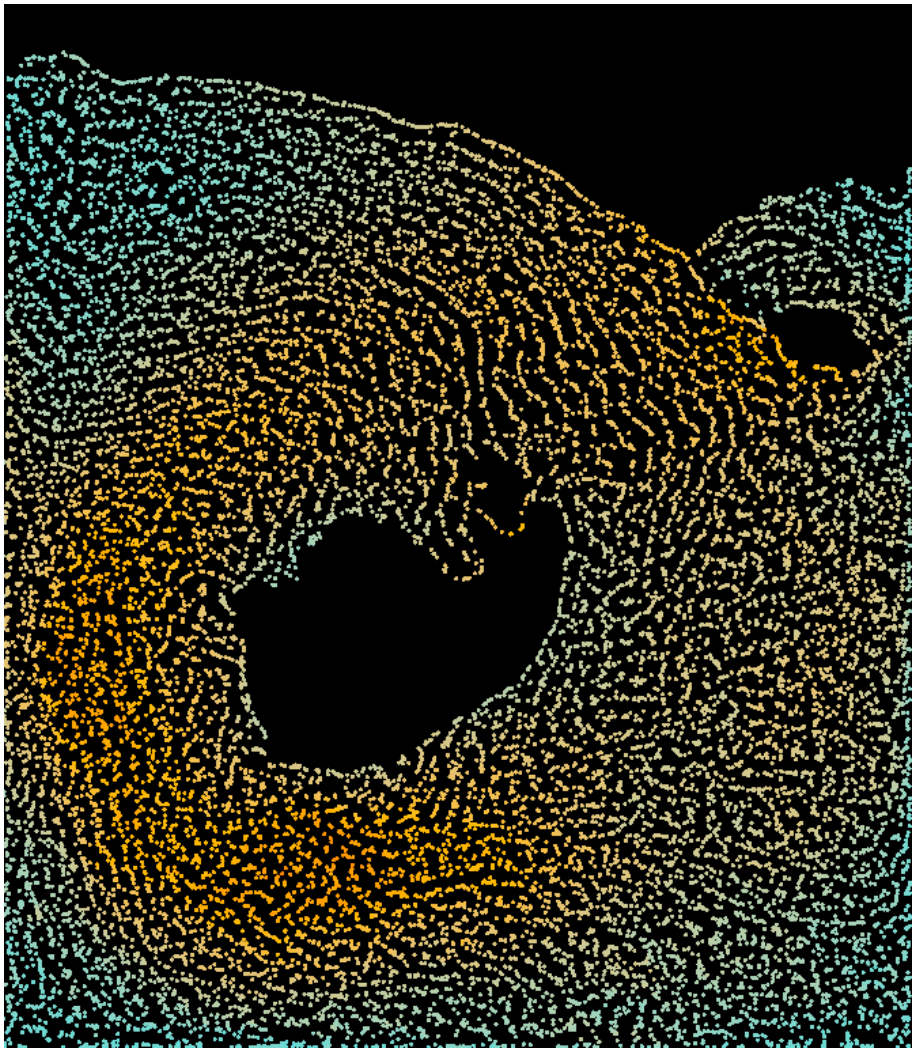
Alfie Kunz

MA395 Essay

Submitted to The University of Warwick

Mathematics Institute

April, 2026



Abstract

This report details the derivation, understanding, and implementation of a highly efficient numerical method for resolving vortical motion in a high Reynolds number flow, that being the hybrid Eulerian-Lagrangian 'Particle-In-Cell' (PIC) model, and its successor, Affine-PIC. These hybrid regimes, require constant interpolation between particle-based velocities (in which advection is performed) and staggered Marker-And-Cell (MAC) grid-based velocities (where divergence-free flow is enforced). The latter is solved by decomposing the grid into a linear system, which can be solved via the Conjugate Gradient Method. We discover that this interpolation between velocity representations introduces a loss of degrees of freedom in the system, and subsequent energy loss in the form of dissipation. This motivates us to 'dope' each particle with a matrix that represents the local velocity gradient. This technique, APIC, conserves angular momentum across the transfers, whilst still allowing for shearing of fluid elements. Comparison to the analytical 'Taylor-Green' vortex yields a Reynolds number $Re = 5420$ (for 64,000 particles divided amongst the fundamental mode, ran in real-time) - a "first of it's kind" result in the field of CFD. Any images shown throughout this essay are captured directly from a computational implementation of these algorithms (written in C# using the Unity Game engine), rendered as a fragment shader using GPU Instancing. To guide this, we also devote some time to the implementation and performance of these methods, such as the aspect of pre-conditioning using the Incomplete Cholesky Factorisation (with Drop Tolerance).

Contents

1 Introduction

The self-advective nature of fluid is perhaps one of the most exciting and deep fields in the current age, with the current explosion in computational processing power allowing for sophisticated and diverse numerical methods. The emergence of vortices in incompressible fluids comes directly from their numerical description through the Navier-Stokes equations, and whose resultant forces are essential for describing even the simplest of fluid dynamics. For example, resolving the D'Alembert paradox for modelling free cylinders in a stream [1], allowing us to study the oscillatory forces which are necessary for the operation of many modern wind turbines [2]. Another key example is the study of vortex shedding to produce aerofoil lift (where even a slight inaccuracy in calculations can result in wild forces on large-scale planes). As such, the criticality of modelling such behaviours, in a way that conserves the natural energy and momentum of the system, cannot be overstated.

In particular, this report places an emphasis on the large scale applicability of these models, as to justify the application to, for example, the aviation industry. We thus search for a versatile, *highly efficient* method, that is able to produce and sustain turbulence that agrees with analytical solutions. After providing the core mathematical and intuitive background of solving a general fluid on the grid, we will derive the PIC method in Section 2, which is able to simulate roughly 30,000 particles in real time (120 simulation steps per second) on a single-core, laptop processor. Whilst these models are built around Euler's Equations [3] (which assumes *inviscid* flow), natural viscosity can arise from the numerical dissipation from particle-to-grid transfers, which breaks our assumptions and limits turbulent motion. Section 3 is therefore dedicated to the discussion and derivation of several techniques (in particular, APIC) that mathematically eliminate certain modes of dissipation. Subsequent discussion, comparison to analytical, inviscid solutions, and reflection, will follow in Sections 4-5.

Alongside being computationally efficient, PIC also scales effectively: as the particle count increases, the interpolation errors for divergence and angular momentum exhibit asymptotically linear time-dependence. Combined with the performance benefits from writing these methods as a GPU compute shader (beyond the scope of this report as it is), we can potentially simulate millions of particles in real-time, making large-scale applications not only effortless but obvious. APIC in particular has made its mark in various industries beyond academia, such as Walt Disney Animation Studios' implementation to simulate highly-detailed crashing waves for 2016's Moana [4].

Alongside this report is my implementation of the below methods in C# (written in the Unity Game engine) whose source code can be found here. Whilst much of this is beyond the scope of this essay, it serves as a 'study tool' for those motivated to contribute further to this field. For the purposes of this report, I will instead reference several algorithms of note.

1.1 Eulerian, Lagrangian, and Hybrid Frameworks

Definition 1.1. *A **Lagrangian** Fluids Simulation Model is one represented by infinitesimal point-particles, each exerting forces on one another as described by the Navier-Stokes equations.*

Definition 1.2. *An **Eulerian** Fluids Simulation Model is one represented by a grid of small cells, each exerting forces on one another as described by the Navier-Stokes equations.*

Naturally, each model has its own distinct advantages. Lagrangian models, for example, thrive in *advection*, as we can simply move particles due to their own velocity (using, for example, Semi-Euler integration)[1]. This also gives them a natural strength in modelling *inviscid* flows. Furthermore, particle representations allow for near effortless computation on the shape and structure of a fluid package - a complicated task for grid-based systems (where we only have access to the *relative* pressures of each cell). On the other hand, 'Finite Difference' Method (FDM) calculations (such as discrete gradient operators in formula (2)) become non-trivial for Lagrangian schemes, giving them a distinct weakness for *projection*[2]. Contrarily, Eulerian models, with fixed cells that mould to FDM calculations effortlessly, thrive in projection and boundary handling.

This motivates the discussion of *hybrid* fluid models, in which we constantly interpolate between particle-based and grid-based approaches, to model the physical aspects in which each respective method thrives. One of the more popular hybrid methods is called 'Particle-In-Cell' (PIC), which dates back to 1955 [6]. As discovered by Jos Stam in 1999 [7], the transfer schemes in PIC dampen noise very effectively (giving us incredible stability), at the cost of exponential energy losses. This report covers two classes of solutions: FLIP [8] (which conserves energy during transfers, at the result of numerical instability) and APIC [9] (which conserves the natural angular momentum of particles during transfers, whilst remaining stable).

As mentioned earlier, we define our Eulerian fluid as a uniform grid of square cells, length s, as to be compatible with FDM calculations. Before moving on, however, it is worth stating that there are other ways of interpreting such fluids, such as the 'Finite Element' method (which has its own specific strengths over FDM, albeit at a large performance cost). In this paradigm, we build a solution ϕ to some PDE (in this case, the Euler Equation) from an asymptotic basis expansion (often polynomial). As we have some choice in this particular basis (where each basis vector represents a specific discretisation of some space / solid boundary), this method excels when handling curved, complex boundaries [10]. However, as our inviscid flow simulation is more concerned with inertial terms, with boundaries consisting almost entirely of square cells, this method would do little other than add a large computational overhead.

Nonetheless, it is still essential to check that FDM algorithms are well-equipped to handle complex solid shapes, such as for accurately simulating the vortex shedding from a cylinder. Inside solid regions, this can be validated by introducing the 'Finite Volume' Method [3], where, for each FDM Cartesian cell $\mathcal{C}$ that the non-trivial solid intersects, we solve the *integral* form of the continuity equation:

$$\oint_{\partial \mathcal{C}} \underline{u} \cdot \underline{n} \, dS = 0.$$

This affects our equations rather simply: instead of using the FDM to form the discrete divergence and pressure operators of each cell $\mathcal{C}$ in (5), $\mathcal{C}$'s contributions are weighted by the 'Area Fraction' that the solid covers that cell. In effect, the solver treats our 'Cut Cell' as having 'less flux effect' than a regular Fluid Cell; therefore representing the sub-grid curvature.

[1] Eulerian models struggle so much with this, in fact, that most models use an approach called *Semi-Lagrangian Advection*, in which we solve for the initial location of some 'virtual' particle that, in timestep Δt, *would have travelled* to a grid cell of interest [5]. We then interpolate this virtual particle's velocity to the grid.

[2] We typically solve this by applying a Gaussian kernel to each particle, creating a low-accuracy approximation of a density field. This forms a pseudo-pressure gradient on particles, which is used to correct the velocity field as in Section 2.2.1.

2 PIC: Particle-In-Cell

2.1 Discrete Operations, and the Staggered Grid

As highlighted by the previous section, velocity fields $\underline{u}(x, y)$, with values known only at discrete grid points, lend themselves nicely to FDM equations. Assuming of $\underline{u}$ sufficiently smooth, we warm up by defining the discrete form of simple differential operators. Beginning with the partial derivative,

$$\frac{\partial \underline{u}}{\partial x}(x, y) = \frac{\underline{u}(x + \frac{s}{2}, y) - \underline{u}(x - \frac{s}{2}, y)}{s}, \tag{1}$$

where s is sufficiently small, we discretise the *divergence*, *gradient*, and *Laplace* operators.

$$(\nabla \cdot \underline{u})(x, y) = \frac{\partial \underline{u}}{\partial x} + \frac{\partial \underline{u}}{\partial y} = \frac{\underline{u}(x + \frac{s}{2}, y) - \underline{u}(x - \frac{s}{2}, y) + \underline{u}(x, y + \frac{s}{2}) - \underline{u}(x, y - \frac{s}{2})}{s}, \tag{2a}$$

$$(\nabla p)(x, y) = \left(\frac{\partial p}{\partial x}, \frac{\partial p}{\partial y}\right) = \frac{p(x + \frac{s}{2}, y) - p(x - \frac{s}{2}, y)\ ,\ p(x, y + \frac{s}{2}) - p(x, y - \frac{s}{2})}{s}, \tag{2b}$$

$$\begin{aligned}(\Delta p)(x, y) &= \frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} \\ &= \frac{\frac{p(x+s,y)-p(x,y)}{s} - \frac{p(x,y)-p(x-s,y)}{s}}{s} + \frac{\frac{p(x,y+s)-p(x,y)}{s} - \frac{p(x,y)-p(x,y-s)}{s}}{s} \\ &= \frac{p(x + s, y) + p(x - s, y) + p(x, y + s) + p(x, y - s) - 4p(x, y)}{s^2}.\end{aligned} \tag{2c}$$

Note that p, unlike $\underline{u}$, is a scalar-valued field. Applied to a *finite, regular grid*, these allow us to describe Euler's equations for an Eulerian fluid. Since the forthcoming Poisson equation (4) equates $\nabla \cdot \underline{u}$ and Δp, it seems natural to *stagger* the velocities for each grid cell via $u_x(x_{i+\frac{1}{2}}, y_j) \mapsto u_x(x_i, y_j)$ and $u_y(x_i, y_{j+\frac{1}{2}}) \mapsto u_y(x_i, y_j)$, and thus store the *pressure* of each cell at each cell's centre. This particular grid is called 'Marker-And-Cell' [11], with representation as depicted in Figure 1, and comes alongside the following notation.

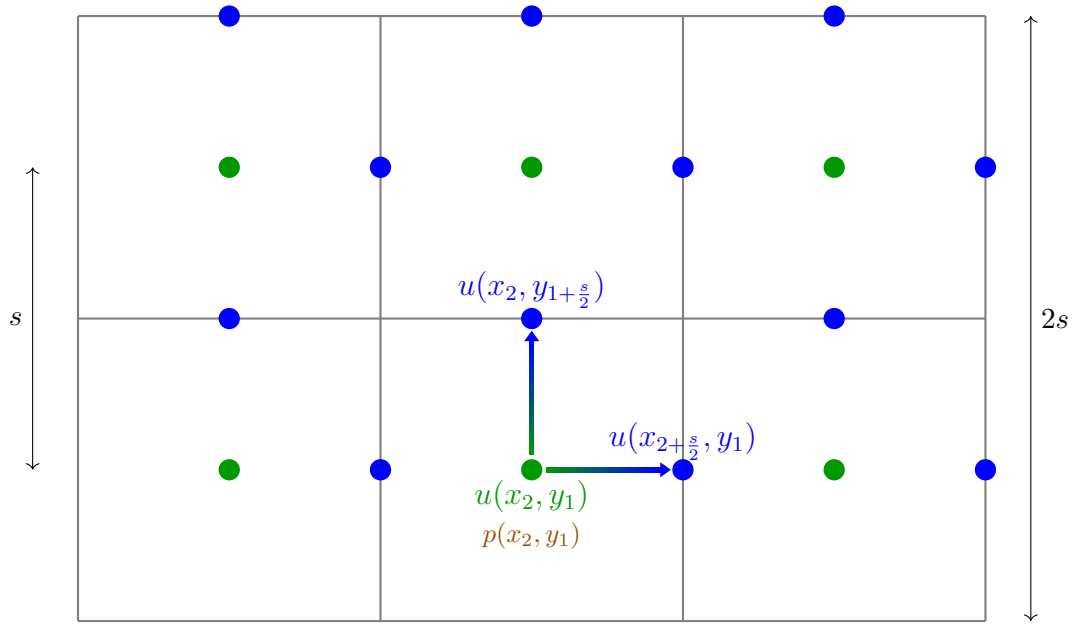


Figure 1: A small-scale example of a 'Staggered MAC' Velocity Grid.

Notation 2.1.

- *We denote* $\mathfrak{i}(a,b) := \{sa,\ s(b+\frac{1}{2})\}$, $\mathfrak{j}(a,b) := \{s(a+\frac{1}{2}),\ sb)\}$ *as the* ***positions*** *of the velocity grid cell, at index* (a,b).
- *We denote* $\mathfrak{u}(a,b) := \{u_x(\mathfrak{i}(a,b))\ ,\ u_y(\mathfrak{j}(a,b))\}$ *as the* ***velocity*** *of the grid cell at* (a,b).
- *We denote* $\mathfrak{p}(a,b) := p(s(a+\frac{1}{2}), s(b+\frac{1}{2}))$ *as the* ***pressure*** *of the grid cell at* (a,b).

This representation of our fluid is now well-handled for enforcing a divergence-free flow.

2.2 Solving for Divergence-Free Velocity

2.2.1 Potentials and Pseudo-Pressures

The primary goal of our solver will be to enforce mass conservation $\nabla \cdot \underline{u} = 0$; first, we discover two (quite remarkable) facts about vector fields: *Helmholtz Decomposition*, and *Scalar Potentials*.

Definition 2.2.

- *A Scalar Potential is some function* $\phi : \mathbb{R}^n \to \mathbb{R}$, *whose gradient* $\nabla\phi$ *gives some vector field* $V : \mathbb{R}^n \to \mathbb{R}^n$.
- *A Vector Potential is some function* $\phi : \mathbb{R}^n \to \mathbb{R}$, *whose curl* $\nabla \times \phi$ *gives some vector field* $V : \mathbb{R}^n \to \mathbb{R}^n$ *[12].*

Theorem 2.3 (Helmholtz Decomposition in $\mathbb{R}^2$)**.** *For some* $V \subseteq \mathbb{R}^2$, ***any*** *vector field* $u \in C^1(V, \mathbb{R}^2)$ *can be written in the form:*

$$u(x,y) = \mathcal{C}(x,y) + \mathcal{D}(x,y),$$

for some $\mathcal{C}, \mathcal{D} \in C^1(V, \mathbb{R}^2)$, *where* $\nabla \times \mathcal{C} = 0$ *and* $\nabla \cdot \mathcal{D} = 0$ *[13].*

To avoid excessive side-tracking, the proof of this theorem is relegated to Appendix B.

Remark 2.4. *It is for this reason that we call solving for a divergence-free flow projection - we are essentially* ***projecting*** *our vector field* $\underline{u}$ *onto the 'divergence-free' basis vector.*

Lemma 2.5 (Gradient of Scalar Potential)**.** *For some open set* $V \subset \mathbb{R}^2$, *let* $\mathcal{C} \in C^1(V, \mathbb{R}^2)$ *be a vector field with the property* $\nabla \times \mathcal{C} = 0$. *Then,* $\exists$ *a Scalar Potential* $\phi(x,y) \in C^1(V, \mathbb{R})$ *with* $(\nabla\phi)(x,y) = \mathcal{C}(x,y),\ \forall (x,y) \in \mathbb{R}^2$.

Proof. $\mathcal{C}$ has the necessary properties to apply Stokes' Theorem [14], from which we get

$$0 = \iint_V \nabla \times \mathcal{C} = \oint_{\partial V} \mathcal{C} \cdot d\underline{s}.$$

Thus, parametrising ∂V via the locally invertible $\gamma(t) : [a,b] \to \mathbb{R}^2$, we can define the scalar potential $\phi : V \to \mathbb{R}^2$ via

$$\phi(\underline{x}) := \int_a^{\gamma^{-1}(\underline{x})} \mathcal{C} \cdot d\underline{s}.$$

$\nabla \times \mathcal{C} = 0$ means γ is arbitrary, and so, defining $f : \mathbb{R} \to \mathbb{R}^2$ with $f(x) = (x, y_0)$ for some y_0 fixed, we get

$$\phi(f(x)) - \phi(f(x_0)) = \int_{f(x_0)}^{f(x)} \mathcal{C} \cdot d\underline{s} = \int_{x_0}^{x} \mathcal{C}_{\S}(f(x'))\ dx'.$$

Finally, applying the Fundamental Theorem of Calculus, and repeating for y, we get

$$\frac{\partial \phi}{\partial x} = \mathcal{C}_x, \quad \Rightarrow \nabla \phi = \left(\frac{\partial \phi}{\partial x}, \frac{\partial \phi}{\partial y} \right) = (\mathcal{C}_x, \mathcal{C}_y) = \mathcal{C}. \qquad \square$$

Remark 2.6. *It is also rather easy to see that the converse applies: given a scalar potential ϕ, $\nabla \times \phi = 0$, and (not relevant for the below methods) given a vector potential $\mathcal{A}$, $\nabla \cdot \mathcal{A} = 0$.*

We delegate the proof of this to Appendix B, and instead use Theorem 2.3 to consider the equation

$$\underline{u}_{new}(x, y) = \underline{u}_{df}(x, y) = \underline{u}(x, y) - \underline{u}_{cf}(x, y) = \underline{u}(x, y) - \nabla p. \tag{3}$$

Taking the divergence yields

$$\nabla \cdot \underline{u} = \Delta p \ + \cancelto{0}{(\nabla \cdot \underline{u}_{df})}, \tag{4}$$

motivating a plan to project our fluid: use its current divergence to calculate the Laplace of some scalar-valued p, then subtract its gradient from $\underline{u}$ to generate a divergent-free flow [15]. In particular, we consider p to be the pressure gradient found in the Navier-Stokes equations. Indeed, a build-up of pressure in some fluid element is directly correlated with a net flux of fluid into it, and thus a violation of mass conservation. However, as we are using $\underline{u}$ to define an 'effective pressure', rather than calculating p explicitly, we call p a *pseudo-pressure gradient*.

We finally consider the boundary condition of defining the pressure of a cell that contains no particles, be it either a solid or air cell. Modelling air as a vacuum[3] enforces $p_{\text{air}} = 0$; a more sensible (and computationally efficient) construction is to restrict the function p to fluid cells *only*, and access air cell pressures, for use in formula (2b), on-the-fly. This has the bonus effect of also restricting $\underline{u}_{df}$, meaning we never have to define the velocities between neighbouring air cells. To tackle solid cells, we make use of the *Neumann boundary condition* [3] $\mathfrak{u}(x, y) \cdot \underline{n} = 0$, for cells (x, y) that neighbour a solid, with normal $\underline{n}$. Via the impermeability boundary condition, this enforces $p_{\text{solid}} = (x, y)$.

2.2.2 Forming a Linear Equation

Applying the discrete operations (2a) and (2c) to equation (4), we are able to define the divergence of some some cell with index (a, b) [15], giving

$$\begin{aligned} \frac{d(a,b)}{s} &:= \frac{\mathfrak{u}_x(a,b) - \mathfrak{u}_x(a-1,b) + \mathfrak{u}_y(a,b) - \mathfrak{u}_y(a,b-1)}{s} \\ &= \frac{\mathfrak{p}(a+1,b) + \mathfrak{p}(a-1,b) + \mathfrak{p}(a,b+1) + \mathfrak{p}(a,b-1) - 4\mathfrak{p}(a,b)}{s^2}. \end{aligned} \tag{5}$$

Denoting F as the set of total cell indices (a, b) in our simulation (of cardinality n_t, at some time t), and C as the set of all cell indices (a, b), we can form the bijective map $f : [\![n_t]\!] \to F$ [4], where $[\![n_t]\!] := \{1, 2, ..., |n_t|\}$. Defining $N(i) : [\![n_t]\!] \to \{0\} \cup [\![4]\!]$ as the number of *non-solid neighbours* the cell with index $f(i)$ has[5], we can apply formula (5) to every fluid cell, to produce a linear equation representation of formula (4):

[3] This seems a reasonable assumption, given that its density is roughly 1000 times less than that of water.

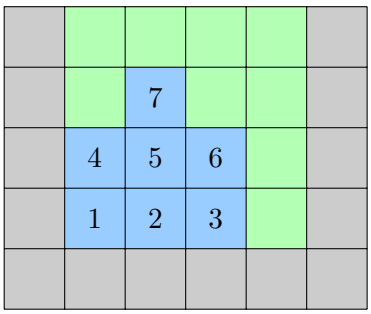
[4] In the following text, we often abuse notation and write $\mathfrak{p}(f(i)) \rightsquigarrow \mathfrak{p}(i)$ and $d(f(i)) \rightsquigarrow d(i)$.

[5] For clarity, we say that the four 'neighbours' of the cell with index (a, b) are those with indices $\{(a+1, b), (a-1, b), (a, b+1), (a, b-1)\}$.

$$-\mathbf{A}\underline{\mathfrak{p}} := -\mathbf{A}\begin{pmatrix}\mathfrak{p}(f(1))\\ \mathfrak{p}(f(2))\\ \vdots\\ \mathfrak{p}(f(n_t))\end{pmatrix} = \begin{pmatrix}d(f(1))\\ d(f(2))\\ \vdots\\ d(f(n_t))\end{pmatrix} =: s\underline{d}, \tag{6}$$

where $\mathbf{A} \in \mathbb{R}^{n_t,n_t}$, referred to as the *Coefficient Matrix*, is defined via

$$A(i,j) = \begin{cases} N(i) & \text{if } i = j \\ -1 & \text{if } f(i) \text{ is a neighbour of } f(j)\ , \\ 0 & \text{else} \end{cases} \tag{7}$$



$$\mathbf{A} = \begin{pmatrix} 2 & -1 & 0 & -1 & 0 & 0 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 & 0 \\ 0 & -1 & 3 & 0 & 0 & -1 & 0 \\ -1 & 0 & 0 & 3 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 4 & -1 & -1 \\ 0 & 0 & -1 & 0 & -1 & 4 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 4 \end{pmatrix}$$

Figure 2: An example in representing a small, grid-based simulation through equation (6), by defining its corresponding Neighbourhood Coefficient Matrix $\mathbf{A}$. Blue colours represents fluid cells, with index $i \in [\![n_t]\!]$, green represents air cells, and grey represents solid cells.

$\mathbf{A}$ therefore represents the *negative discrete Laplace Operator* $-\Delta$, which will become increasingly sparse as n_t grows. We prove the following properties.

Lemma 2.7. *The matrix* $\mathbf{A}$*, as defined in* (7)*, is symmetric, and positive-definite when restricted to multiplication by non-constant vectors.*

Proof. Symmetry follows immediately from the definition of $\mathbf{A}$ and a 'neighbour' (see comment 5). Recall that $\mathbf{M} \in \mathbb{R}^{n,n}$ is positive-definite if $\underline{p} \cdot \mathbf{M}\underline{p} > 0$, $\forall \underline{p} \in \mathbb{R}^n \setminus \{0\}$. Applying this to $\mathbf{A}$ gives:

$$\underline{p} \cdot \mathbf{A}\underline{p} = \sum_{i=1}^{n_t} p_i \left(N(i)p_i - \sum_{\text{Nbs(i)}} p_j \right) = \sum_{i=1}^{n_t} \left(N(i)p_i^2 - \sum_{\text{Nbs(i)}} p_i p_j \right),$$

where $\text{Nbs}(i) := \{j \in [\![n_t]\!] \mid f(i) \text{ is a neighbour of } f(j)\}$. As $|\text{Nbs(i)}| = N(i)$, for each neighbour pair p_a and p_b we can bijectively assign a p_a^2 from $N(a)p_a^2 = \underbrace{p_a^2 + p_a^2 + \ldots}_{N(a)}$, and a p_b^2 from $N(b)p_b^2$, to form

$$p_a^2 \ \overset{\text{i=a loop}}{-\ \overbrace{p_a p_b}}\ \overset{\text{i=2 loop}}{-\ \overbrace{p_b p_a}}\ + p_b^2 = (p_a - p_b)^2.$$

From this, we can remove the neighbour pair (a, b) from the summation loops to get

$$\underline{p} \cdot \mathbf{A}\underline{p} = \sum_{i=1}^{n_t} \left(N(i)p_i^2 - \sum_{\text{Nbs(i)}} p_i p_j \right) + (p_a - p_b)^2.$$

Repeating for all neighbour pairs (i, j) (decrementing $N(a)$, $N(b)$ for each pair) sets $N(i) = 0\ \forall i$

via our bijective assignment. Under non-constant vectors p, we get the strictly positive quantity,

$$\underline{p} \cdot \mathbf{A}\underline{p} = \sum_{\text{neighbours } (i,j)} (p_i - p_j)^2. \qquad \square$$

Remark 2.8. *From this, we can see that* $\underline{p} \cdot \mathbf{A}\underline{p} = 0$ *only if* $\exists \alpha \in \mathbb{R}$ *s.t* $p_i = \alpha,\ \forall i \in \mathbb{R}^n$ *(ie: p is a constant vector). We are safe to ignore this for our simulation, as equation (4) reveals that such a case would result in* $\underline{u}$ *already being incompressible - no computation is required.*

As our matrix $\mathbf{A}$ is in this pleasant form, and is sparse, we employ a very powerful technique to solving the linear equation (6): the Conjugate Gradient Method.

2.2.3 The Conjugate Gradient Method

As a reminder, we are aiming to solve the linear equation

$$\mathbf{A}\underline{\mathfrak{p}} = -s^2\underline{d} =: \underline{b}, \tag{8}$$

where $\mathbf{A}$ is the symmetric, positive-definite, and sparse matrix with construction as in formula (7), and $\underline{d}$ as in formula (5). Denoting $\underline{u} \cdot \underline{v}$ as the standard Euclidean dot product, we say:

Definition 2.9. *Let* $\mathbf{A} \in \mathbb{R}^{n_t, n_t}$ *We call vectors* $\underline{u}, \underline{u} \in \mathbb{R}^{n_t}$ *conjugate if the inner product with respect to* $\mathbf{A}$*,*

$$\langle \underline{u}, \underline{v} \rangle_A := \underline{u} \cdot \mathbf{A}\underline{v},$$

is equal to 0*. In other words, they are orthogonal through A.*

(You may want to convince yourself that, through the properties of $\mathbf{A}$, $\langle \underline{u}, \underline{v} \rangle_A$ is a well-defined inner product [16].) Note that $X := \{\underline{x}_1, \underline{x}_2, \ldots, \underline{x}_{n_t}\}$, some set of mutually conjugate vectors, forms a basis for $\mathbb{R}^{n_t}$, and so we can express equation (8) via:

$$\underline{b} = \mathbf{A}\underline{\mathfrak{p}} = \sum_1^{n_t} \alpha_i \mathbf{A}\underline{x}_i,$$

with coefficients $\alpha_i \in \mathbb{R}$ [3]. Pre-multiplying this by some $\underline{x}_k$ gives the summation $\sum_1^{n_t} \alpha_i \langle \underline{x}_i, \underline{x}_k \rangle_A = \alpha_k \langle \underline{x}_k, \underline{x}_k \rangle_A$, by orthogonality. Using this, we can define these coefficients as

$$\alpha_k = \frac{\underline{x}_k \cdot \underline{b}}{\langle \underline{x}_k, \underline{x}_k \rangle_A}. \tag{9}$$

We remark that, if we can find some mutually conjugate vector space (consisting of X's elements, and their corresponding coefficients α_i), then we can receive an expression for $\underline{b}$, thus solving the system [3]. Simple as this logic is, the large cardinality of X ($= n_t$, the number of fluid cells) renders this intractable. Rather, it is more elegant to pick each x_i inductively, such that we still generate a *reasonable* approximation for $\underline{b}$ when stopping after $m < n_t$ iterations.

Exploiting the fact that $\mathbf{A}$ represents a (negative) discrete Laplace operator, we know that $\exists f : \mathbb{R}^n \to \mathbb{R}$ with Hessian $\mathbf{A}$: we propose the *quadratic form* [17], given by

$$f(\underline{\mathfrak{p}}) = \frac{1}{2}\underline{\mathfrak{p}} \cdot \mathbf{A}\underline{\mathfrak{p}} \ - \ \underline{\mathfrak{p}} \cdot \underline{b}. \tag{10}$$

We are therefore attempting to minimise $f(\underline{\mathfrak{p}})$, illustrated through the contours in Figure 3. As $\mathbf{A}$ is positive-definite, $f(\underline{\mathfrak{p}}_\infty)$ (where $\underline{\mathfrak{p}} = \underline{\mathfrak{p}}_\infty$ is the solution of equation (8)) is a strict local minimum:

$$\nabla f(\underline{\mathfrak{p}}_\infty) \;=\; \frac{1}{2}\mathbf{A}^T\underline{\mathfrak{p}}_\infty + \frac{1}{2}\mathbf{A}\underline{\mathfrak{p}}_\infty - \underline{b} \quad = \; \mathbf{A}\underline{\mathfrak{p}}_\infty - \underline{b} \;=\; \underline{0}.$$

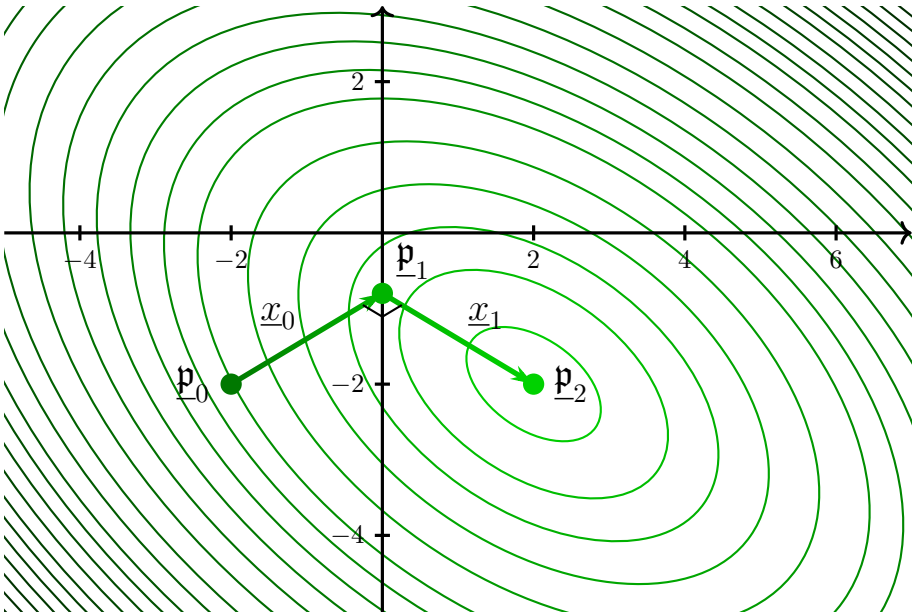


Figure 3: A visualisation of the Conjugate Gradient Method in the plane (ie: $n_t = \mathbb{R}^2$), where each ellipse represents constant values (contours) of the quadratic form, as in (10). Resultantly, our method gives $\underline{\mathfrak{p}}_2 = \underline{\mathfrak{p}}_\infty$.

Inspired by this, we employ the *gradient descent* technique, which aims to minimise f, given some initial 'guess' $\underline{\mathfrak{p}}_0$ (as measured by a residue, $\underline{r}_0 = -\nabla f(\underline{\mathfrak{p}}_0)$). As this is now an error term, we write (9) as

$$\alpha_k \to \alpha(\underline{r}_k) = \frac{\underline{x}_k \cdot \underline{r}_k}{\langle \underline{x}_k, \underline{x}_k \rangle_A}.$$

We must also ensure that (by the definition of X) in each gradient descent iteration, the vector direction in which we descend ($\underline{x}_k$) is mutually conjugate to all other $\{\underline{x}_1, \ldots, \underline{x}_{k-1}\}$. This is ensured via the *Gram-Schmidt Orthonormalization* process [3, 16], by defining:

$$\underline{x}_k = \underline{r}_k - \sum_{i<k} \frac{\langle \underline{x}_i, \underline{r}_k \rangle_A}{\langle \underline{x}_i, \underline{x}_i \rangle_A} \, \underline{x}_i, \tag{11}$$

or rather, removing any vector projections in all previous $\underline{x}_i \in X$. This gives the solution

$$\underline{\mathfrak{p}}_{k+1} = \underline{\mathfrak{p}}_k + \alpha_k \underline{x}_k, \quad \text{with error} \;\; \underline{r}_{k+1} = \underline{b} - \mathbf{A}\underline{\mathfrak{p}}_{k+1}.$$

Under this constructed basis $\underline{x}_k$, we see that $\lim_{k \to n_t} \underline{\mathfrak{p}}_k = \underline{\mathfrak{p}}_\infty$ (JR Shewchuk [17] details a rigorous proof of this), and so we can terminate our algorithm as $||\underline{r}||_\infty := \max\{r_1, \ldots, r_{n_t}\}$ passes below some threshold. Note that we use the l_∞ norm rather than the standard l_2 norm [3] - we want to correct the *worst-case* areas of divergent flow; otherwise we might have realistic fluid in an averaged sense, apart from some small areas that (locally) behave poorly.

This solver requires some initial guess to function – for this, we save the pressure values for each cell after every iteration, such that when we run the next iteration and save our list of fluid cells,

we can retrieve said pressure values and store them as an initial guess [3]. Ie: $\underline{\mathfrak{p}}_0^t = \underline{\mathfrak{p}}_\infty^{t-1}$. In my testing, this reduces the number of iterations roughly 20%, compared with setting $\underline{\mathfrak{p}}_0^t = \underline{0}$.

2.2.4 Preconditioning

It may be that the Conjugate Gradient algorithm, performed on the linear system $\mathbf{M}^{-1}\mathbf{A}\underline{\mathfrak{p}} = \mathbf{M}^{-1}\underline{b}$ (where $\mathbf{M}$ is some symmetric, positive-definite matrix in $\mathbb{R}^{n_t,n_t}$), converges much faster than the standard formula (8). Trivially, a single iteration is required if $\mathbf{M} = \mathbf{A}$. Hence, it is often a good use of computation time to first construct a *pre-conditioner* $\mathbf{M}$, as a viable approximator of $\mathbf{A}$, for use in solving the *Preconditioned* Conjugate Gradient (PCG) Method [17].

The CG algorithm converges well if $\mathbf{A}$'s eigenvalues are packed together, meaning the quadratic form f in (10) is locally steep about $\underline{\mathfrak{p}}_\infty$ [3]. This emerges for particularly laminar, viscous flow (where $\mathfrak{p}$ varies slowly between neighbouring cells). Hence, $\mathbf{M}$ is a good choice for our pre-conditioner if it is sparse and easy to compute from $\mathbf{A}$, yet is such that $\mathbf{M}^{-1}\mathbf{A}$'s eigenvalues are all close in value (and to 1). There are many ways to construct such a pre-conditioner: we will follow R. Bridson's recommendation [3] of '(Modified) Incomplete Cholesky Factorisation with (Drop-Tolerance) Thresholding, Level-Zero (MICT-0)'. Other schemes may also be used, but comparisons between these go beyond the scope of this essay.

Let the symmetric $\mathbf{M}$ be factorised into the lower & upper triangular matrix product $\mathbf{L}\mathbf{L}^T$, and consider the linear equation,

$$(\mathbf{L}^{-1}\mathbf{A}[\mathbf{L}^{-1}]^T)(\mathbf{L}^T\underline{\mathfrak{p}}) = \mathbf{L}^{-1}\underline{b},$$

which has identical solutions to that of (8). As $(\mathbf{L}^{-1}\mathbf{A}[\mathbf{L}^{-1}]^T)$ is symmetric and positive-definite, we can apply the CG Method as before, but now, after constructing $\underline{r}_k$, we *precondition* to get $\underline{z}_k$, the vector which solves,

$$\mathbf{M}\underline{z}_k = \underline{r}_k, \tag{12}$$

and use $\underline{z}_k$, rather than $\underline{r}_k$, for calculating $\underline{x}_k$ in formula (11). Recalling that $\mathbf{M}$ is chosen to be easily invertible, solving these sub-systems is incredibly efficient: rearranging equation (12) gives,

$$\underline{z}_k \;=\; \mathbf{M}^{-1}\underline{r}_k \;=\; (\mathbf{L}^{-1})^T\mathbf{L}^{-1}\underline{r}_k \;=\; (\mathbf{L}^{-1})^T\underline{a}_k \;; \qquad \underline{a}_k := \mathbf{L}^{-1}\underline{r}_k,$$

which can be solved in turn by finding solutions for the below sub-equations:

$$\underline{r}_k = \mathbf{L}\underline{a}_k \qquad \text{(solve for } \underline{a}_k\text{)}, \tag{13a}$$

$$\underline{a}_k = \mathbf{L}^T\underline{z}_k \qquad \text{(solve for } \underline{z}_k\text{)}. \tag{13b}$$

Equation (13a) can be solved using the basic *forward* substitution technique (as $\mathbf{L}$ is lower triangular), and equation (13b) can be solved similarly using *backward* substitution [3].

Finally, we consider the MICT-0 factorisation of $\mathbf{M} = \mathbf{L}\mathbf{L}^T$, and thus the task of 'defining' our specific pre-conditioner $\mathbf{M}$. Motivated by our previous analysis, we *approximate* $\mathbf{L}$ via some lower triangular matrix $\mathbf{K}$, which retains $\mathbf{A}$'s sparsity, but still well-approximates it, with $A \approx M := KK^T$ positive-definite. The 'thresholding' technique in this factorisation (for which we will use standard *Drop-Tolerance*) defines the critical value τ under which $\mathbf{A}$'s entries are deemed to have a negligible effect on the description of the fluid, and can thus be zeroed in order

to preserve sparsity [18]. We compute $\mathbf{K}$ via the following algorithm [18, 19]:

$$\mathbf{K}_{ij} = \begin{cases} \sqrt{w\mathbf{K}_{ii}} & i = j \\ w & \text{if } j < i,\ |w| \geq \tau \\ 0 & \text{if } j < i,\ |w| < \tau \\ 0 & j > i \end{cases}; \qquad \text{where } w = \frac{\mathbf{A}_{ij} - \sum_{k=1}^{j-1} \mathbf{K}_{ik}\mathbf{K}_{jk}}{\mathbf{K}_{jj}}.$$

See here for my implementation of this Preconditioned Gradient Method, in my simulation. Note that this is one of many possible optimisations we can apply to standard CGM. For the purposes of space, an intrigued reader can continue this discussion in Appendix A.2.

2.3 Bilinear Interpolation

To transfer velocities between particles and the grid, we use bilinear interpolation' [15], which can be represented via the respective linear operators $\underline{u}_{\text{grid}} = \text{BiLerpPG}_{\underline{x}_\text{p}}(\underline{u}_\text{p})$ (for particles to grid) and $\underline{u}_\text{p} = \text{BiLerpGP}_{\underline{x}_\text{p}}(\underline{u}_\text{g})$, where $\underline{x}_p$ denotes particle positions (and $\underline{x}_{\text{grid}}$ are pre-determined via the MAC format). Considering BiLerpPG, for example, we first assign eight *weights* $\{w_p^1, w_p^2, ..., w_p^8\}$ for each particle p, each holding the relative (ie: $w^i \in [0, 1]$) distances from $\underline{x}_p$ to some specific point on the grid. In our case, $\{w^1, ..., w^4\}$ represents the relative distances to p's four closest $\mathfrak{u}_x$ locations on the grid, and $\{w^5, ..., w^8\}$ represents the relative distances to the four closest $\mathfrak{u}_y$ locations. We can see this visually in Figure 4.

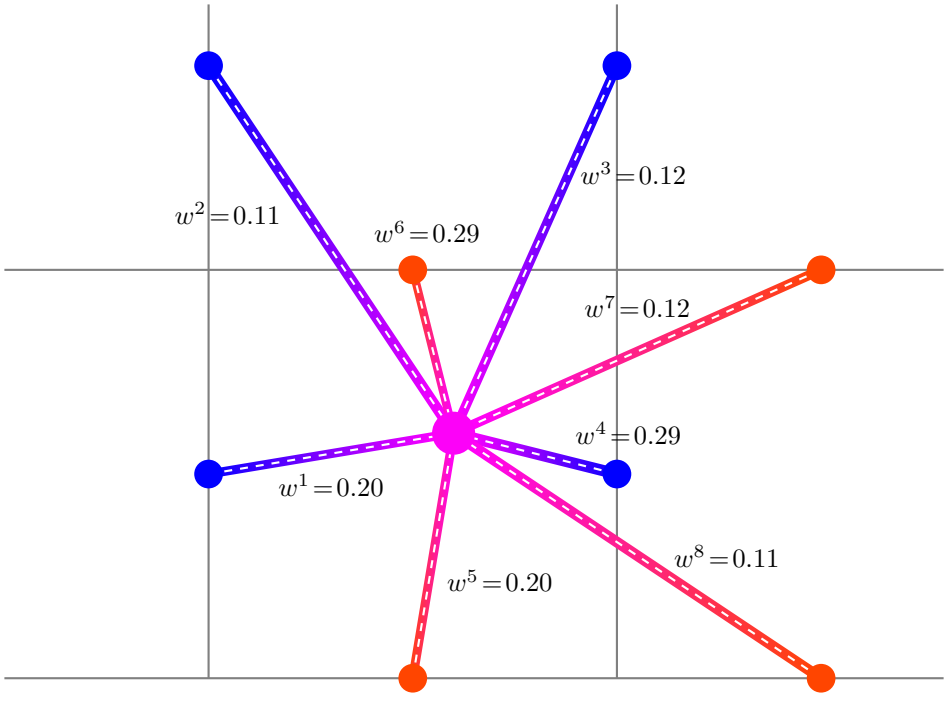


Figure 4: Construction of weights w_p^i for Bilinear Interpolation (Particle-to-Grid) on a MAC Grid. The blue nodes represent the MAC x-velocity locations $\{w^1, ..., w^4\}$, and the orange nodes represent the y-velocity locations $\{w^5, ..., w^8\}$, with the pink particle at the centre representing the location of $\underline{x}_p$.

Mathematically, we can calculate these weights as such:

$$w_p^{1,5} = \frac{\Delta x}{s}\frac{\Delta y}{s}, \qquad w_p^{2,6} = \frac{\Delta x}{h}\left(1 - \frac{\Delta y}{s}\right),$$

$$w_p^{3,7} = \left(1 - \frac{\Delta x}{s}\right)\left(1 - \frac{\Delta y}{s}\right), \qquad w_p^{4,8} = \left(1 - \frac{\Delta x}{s}\right)\frac{\Delta y}{s},$$

where Δx and Δy represent the horizontal and vertical distance from the particle to the bottom-left MAC velocity component of question, and s again denotes cell width [3]. Taking a weighted average of p's weights (to conserve linear momentum) yields the interpolation equation:

$$\mathfrak{u}_x(a,b) = \frac{\sum_{1\le i\le 4}\sum_p w_p^i\ \underline{u}_p}{\sum_{1\le i\le 4} m_i^x}, \qquad \mathfrak{u}_y(a,b) = \frac{\sum_{5\le i\le 8}\sum_p w_p^i\ \underline{u}_p}{\sum_{5\le i\le 8} m_i^x}, \tag{14}$$

where we define $m_i^x = \sum_p w_p^i$ and $m_i^y = \sum_p w_p^i$ as the 'mass effect' of p on the grid node i. In the discussion to follow, equations (14) are combined to form the single interpolation equation:

$$\underline{u}_{\text{grid}} = \frac{\sum_i \sum_p w_p^i\ \underline{u}_p}{\sum_i (m_i := \sum_p w_p^i)}. \tag{15}$$

A similar process can also formed for BiLerpGP. Also note that, via our Dirichlet boundary conditions, we must enforce $w_p^{solid} = 0$: no particle velocity can be interpolated to a solid cell.

This concludes our analysis of the PIC method. Elegant as it is, there are several considerations one must make when implementing this computationally. In particular, how do we efficiently store our sparse matrices, such as those in equation (7)? How specifically do we *advect* our fluid? For space constraints, however, this discussion is relegated to Appendix A, which contains key discussions on improving the accuracy and speed of our solver.

3 Vortex Analysis

We begin with a brief reflection on what we have covered thus far, in deriving PIC - specifically, we consider the emergence of vortex behaviour, and then dive into the angular momentum properties which drive it. This will motivate the need to 'upgrade' our model with new energy modes, or velocity gradient information, which allow for better deformation of fluid elements. Finally, we will quantify the total angular momentum of our fluid, in order to compare methods.

3.1 Problems with PIC

With our Particle-In-Cell method now in place, one may be surprised to see a lack of vortex motion! In fact, the fluid as a whole appears rather smooth and 'gelatinous'. This can initially seem strange, as we are specifically considering inviscid, high Reynolds number flows, in which we can drop viscosity term $\nu\nabla^2\underline{u}$ in the Navier-Stokes equations; why is the fluid behaving so viscously? The answer: *numerical dissipation*. An eagle-eyed reader may have noticed a fatal flaw in our hybrid models - we are constantly *smoothing* our particle and grid velocities when interpolating to the other respective scheme [20]. This averaging nature results in a large system energy loss, presented in the form of incredibly high viscosity, and thus a lack of vortices [20].

Written another way, any artificial noise added to particle velocity is filtered out via the respective transfer schemes. The physical interpretation of this lies in the system's *degrees of freedom* (DoF) - we allocate 4 particles to each grid cell (with 8 total DoF), the information of which is then condensed into the grid's x and y velocity components - 2 DoF. Ie: the bilinear operator has a kernel of dimension 4 [21], producing the 'smoothing' effect we are wishing to rectify.

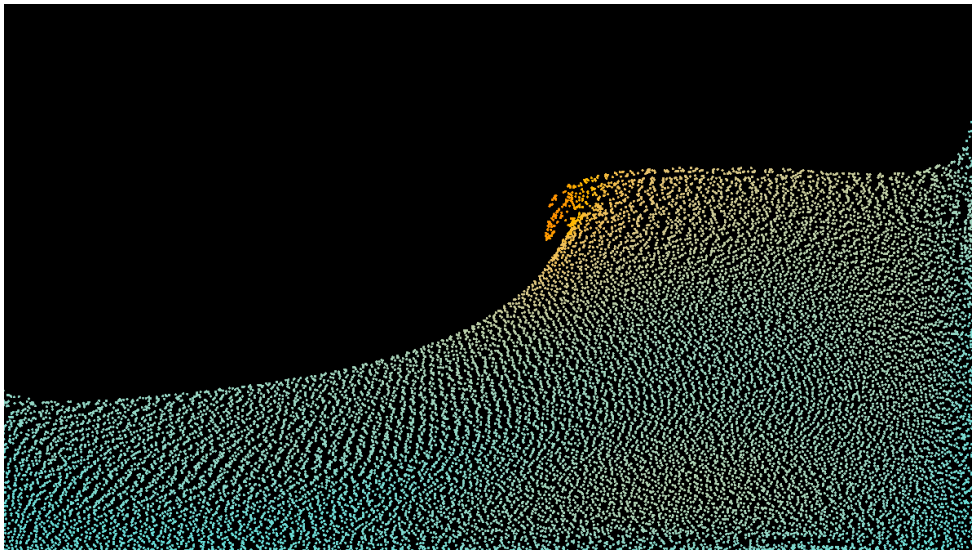


Figure 5: A Low-Resolution, PIC Simulation - note the smoothing of the velocity field.

We consider two approaches to remedying this uninspiring behaviour: *FLIP (Fluid-Implicit Particle)*, whose aim in reducing numerical dissipation is set by conserving energy in particle-grid transfers, and *APIC (Affine Particle-In-Cell)*, which attempts to quantify and represent angular momentum losses of particles in the grid representation.

3.2 FLIP: Fluid-Impicit Particle

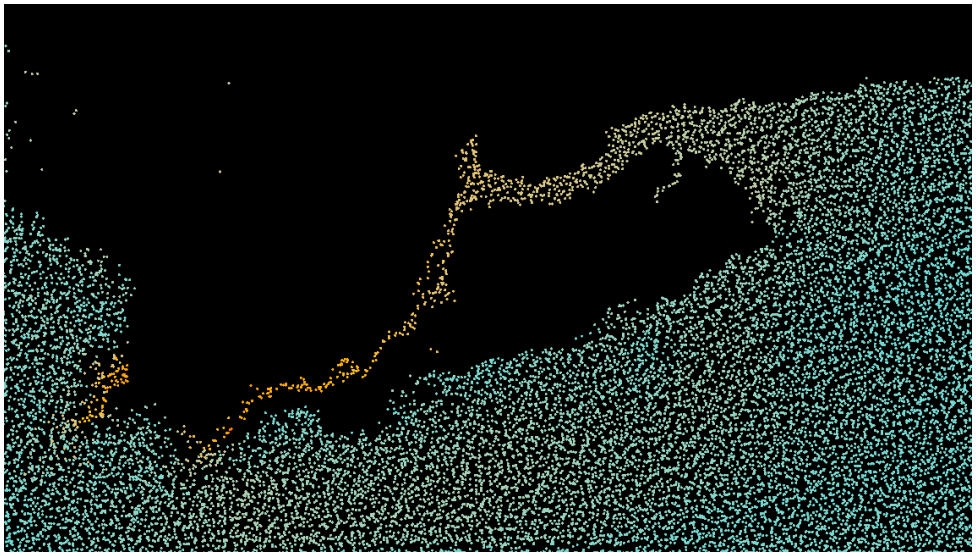


Figure 6: A Low-Resolution, FLIP Simulation - note the lack of smoothing on the left side, where particles of vastly different velocities are located very close together.

FLIP is, by far, the simplest of the two methods, and works as such: we interpolate from the particles to the grid as normal, then make a *full copy* of the grid velocities before any work is undergone. Once we have our new grid velocities (modified due to projection) to transfer back to the particles, we instead interpolate the *change in grid velocity*, adding this to the particles' current velocities. This effectively 'removes' a smoothing step, and hence a large source of our numerical dissipation [8]. One can show that this scheme conserves the internal energy and total momentum across the transfers, and that the loss of kinetic energy is only non-negligible when the velocity distributions of particles is poorly represented by a bilinear interpolation function.

This is the case only for strongly accelerated flow, where our solver struggles to interpret $\nabla \underline{u}$ accurately [8, 22]. As our bilinear interpolation operator has a non-trivial kernel, it is possible for some artificial noise added to the particles in some cell to not be transferred to the grid; by our FLIP scheme, no correction / dampening term will be added back to the particles. Hence, FLIP is an inherently *unstable* scheme [21]. We can see this numerical instability in Figure 6.

Comfortingly, we can actually interpolate between the PIC and FLIP models [3], in an attempt to get the 'best of both worlds', and fine-tune the viscosity of the system: we define the *blending factor* $\alpha \in [0, 1]$, and update particle velocities via a superposition of interpolations:

$$\underline{u}_p = \alpha \cdot \underbrace{\text{BiLerpGP}_{\underline{x}_p}(\underline{u}^{new}_{grid})}_{\text{PIC}} + (1-\alpha) \cdot \underbrace{\left[\underline{u}^{old}_p + \text{BiLerpGP}_{\underline{x}_p}(\underline{u}^{new}_{grid} - \underline{u}^{old}_{grid})\right]}_{\text{FLIP}}.$$

One can relate α to the *kinematic viscosity* of the fluid, ν, via $\alpha = 6\nu\Delta t/\Delta x^2$ [3], from which we can follow the well-supported value of $\alpha = 0.05$ - a good approximator of the natural viscosity of water [3]. This produces the most visually pleasing and intuitively accurate simulation we have seen thus far in Figure 7, and finally allows for perceptible vortex motion in a stable environment.

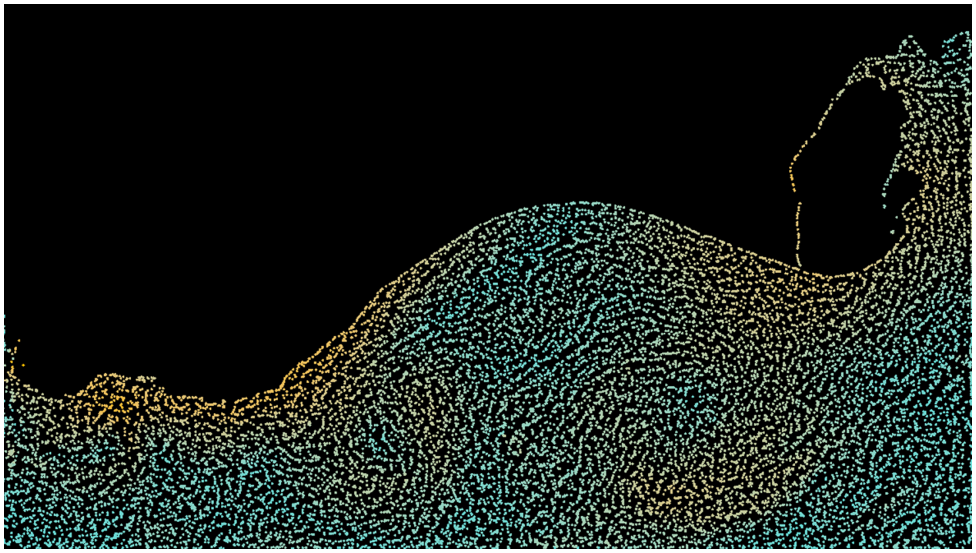


Figure 7: A Low-Resolution, PIC/FLIP Simulation ($\alpha = 0.05$) - this greatly improves upon PIC with little effort, allowing for shearing and rotational deformation whilst inheriting (for the most part) the stability of PIC.

We see that, whilst FLIP preserves energy across time-steps, and thus rotational motion, necessitating *any* amount of PIC ($\alpha > 0$) in order to retain stability ensures that angular momentum is not conserved across multiple simulation steps.

3.3 APIC: Affine Particle-In-Cell

3.3.1 Conservation of Angular Momentum

From the previous discussion, it is rather clear how we can reduce the numerical dissipation of our transfer operators, without resorting to unstable schemes - add more DoF to our grid representation [21] (through the BiLerpGP operator, in particular). Specifically, we will learn how to 'dope' each cell with a linear operator (representing the angular momentum loss in the particle representation) in a way that does not affect our Conjugate Gradient Algorithm.

First, however, we aim to derive the amount of momentum we lose via our particle-grid transfers. This is specifically restricted to the *angular* kind - our bilinear interpolation operators (15) by construction conserve linear momentum [9]. At some time t, let our MAC grid have divergence-free (as in, post projection) velocity components $\underline{u}_g$ at fixed positions $\underline{\chi}_g$, and thus weights w_p^g from some particle p. Formally, these are given by the collection of grid nodes $g \in G$, with velocities

$$\left\{ \begin{pmatrix} \mathfrak{u}(a,b)_x \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ \mathfrak{u}(a,b)_y \end{pmatrix} \right\}, \quad \text{at positions } \{\mathfrak{i}(a,b), \mathfrak{j}(a,b)\}.$$

Then, we can form an equation for the angular momentum of each of our representations,

$$L_P = \sum_p \underline{x}_p \times \underline{u}_p, \qquad L_G = \sum_g \underline{\chi}_g \times m_g \underline{u}_g = \sum_g \sum_p w_p^g (\underline{\chi}_g \times \underline{u}_p), \tag{16}$$

where $\sum_p$ is over all particles within the interpolation range of the grid cell g, and $m_g \underline{u}_g = \sum_i \sum_p w_p^g \underline{u}_p$ is a restated form of equation (15) [9]. Also note that, as $\underline{x}, \underline{u}$ live in $\mathbb{R}^2$, L_P and L_G are scalar values. The angular momentum loss for some particle p in the BiLerpGP operator, therefore, compared to its representation on the grid, comes directly from 'pivoting' it around each grid point g (with position $\underline{\chi}_g$) it influences, and crossing this with that grid point's momentum:

$$L_p^{\text{loss}} = \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p^{\text{old}}) \times \underline{u}_{\text{g}}, \tag{17}$$

where $\underline{x}_p^{\text{old}}$ is p's position *before* advection (as opposed to $\underline{x}_p$, which is defined afterwards). Trivially, our interpolation operators conserve momentum if the particles are placed directly on top of the grid points, and errors accumulate linearly as the particles begin to distribute. We will thus attempt to enrich our particle velocities with these L_p^{loss}'s, in the attempt to induce them with more DoF [23] (in other words, taking the loss in angular momentum in BiLerpGP, and feeding this into the interpolation velocities for the next simulation step's BiLerpPG, thus locally conserving the quantity for *both* operators) [9].

Definition 3.1. *We call some function $f : X \to Y$ locally... [21]*

- ***constant*** *if, $\forall x \in X$, $\exists$ an open neighbourhood $U \subset X$, containing x, such that f restricted to U is constant: $f(u) = f(x)$, $\forall u \in U$.*
- ***affine*** *if, $\forall x \in X$, $\exists$ an open neighbourhood $U \subset X$, containing x, such that f restricted to U is linear: $f(u) = f(x) + \mathbf{L}(u - x)$, $\forall u \in U$, and where $\mathbf{L} \in \mathcal{L}(X, Y)$.*
- ***rigid*** *if it is affine, with the extra condition that L is anti-symmetric ($\mathbf{L}^T = -\mathbf{L}$).*

In these PIC transfer operations, we essentially treat each particle as *locally constant* (where this 'local' behaviour is apt on the scale of the size of a grid cell). We first introduce the Rigid Particle-In-Cell improvement, which treats each particle as *locally rigid*.

3.3.2 Rigid Approximations

From Definition 3.1, we can think of a locally rigid function as one which *preserves distances* in some open neighbourhood $U \ni x$: $|f(u) - f(x)| = |u - x|$, $\forall u \in U$. This term *rigid* comes from rigid-body physics, in which we assume that some small body (or fluid element) cannot deform.

Attaching a locally rigid velocity field $u \in C^2(\mathbb{R}^2, \mathbb{R}^2)$ to each particle p at position $\underline{x}_p$ gives

$u(\underline{x}) = u(\underline{x}_p) + \mathbf{R_p}(\underline{x} - \underline{x}_p)$, where $\mathbf{R_p} \in \mathcal{L}(\mathbb{R}^2, \mathbb{R}^2)$ is anti-symmetric, and represents a local approximation of ∇u [23]. Relating this to the physics of combining linear and angular momentum, we can relate $\mathbf{R_p}$ to an angular velocity operator, and thus write,

$$\boldsymbol{\omega}_p^* = \mathbf{R_p}, \qquad \text{where} \quad \boldsymbol{\omega}_p^* a = \underline{\omega_p} \times \underline{\alpha} = \omega \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \alpha_x \\ \alpha_y \end{pmatrix}. \tag{18}$$

Generally, we relate angular velocity to angular momentum via the equation $\underline{L}_p = \mathbf{I}_p \underline{\omega}_p$, where

$$(\mathbf{I}_p)_{ij} := \sum_k m_k (r_k^2 \delta_{ij} - (x_k)_i (x_k)_j)\, dV \qquad \in \mathbb{R}^{3,3}, \tag{19}$$

is the inertia tensor (representing a body's natural resistance to rotation) [24, Equations 5-6, 5-8] of the rigid body χ that p contributes to during interpolation - that being the family of MAC velocity component points at locations $\underline{\chi}_g$, with weights $\{w_p^1, \dots, w_p^8\}$. As ω and L are scalar values (representing a vector perpendicular to $\underline{u}_p$), we can write equation (19) as

$$\underbrace{I_p := I_{zz}}_{\cong (x_{p\to g}^2 + y_{p\to g}^2)} = \sum_g w_p^g || \underbrace{\underline{\chi}_g - \underline{x}_p}_{\underline{\Delta x}^{gp}} ||^2 = \sum_g w_p^g \left[(\underline{\Delta x}_x^{gp})^2 + (\underline{\Delta x}_y^{gp})^2 \right]. \tag{20}$$

Hence, following C. Jiang [9] by combining equations (17) and (20) gives,

$$\omega_p = \frac{L_p^{\text{loss}}}{I_p} = \frac{\sum_g w_p^g \left[(\underline{\chi}_g - \underline{x}_p^{\text{old}}) \times \underline{u}_g \right]}{\sum_g w_p^g ||\underline{\chi}_g - \underline{x}_p||^2}, \tag{21}$$

and so the $\sum_p w_p^i \underline{u}_p$ term in equation (15) becomes

$$\sum_p w_p^i \underline{u}_p^{\text{rigid}}, \qquad \text{where} \quad \underline{u}_p^{\text{rigid}} = \underline{u}_p + \boldsymbol{\omega}_p^* (\underline{\chi}_i - \underline{x}_p). \tag{22}$$

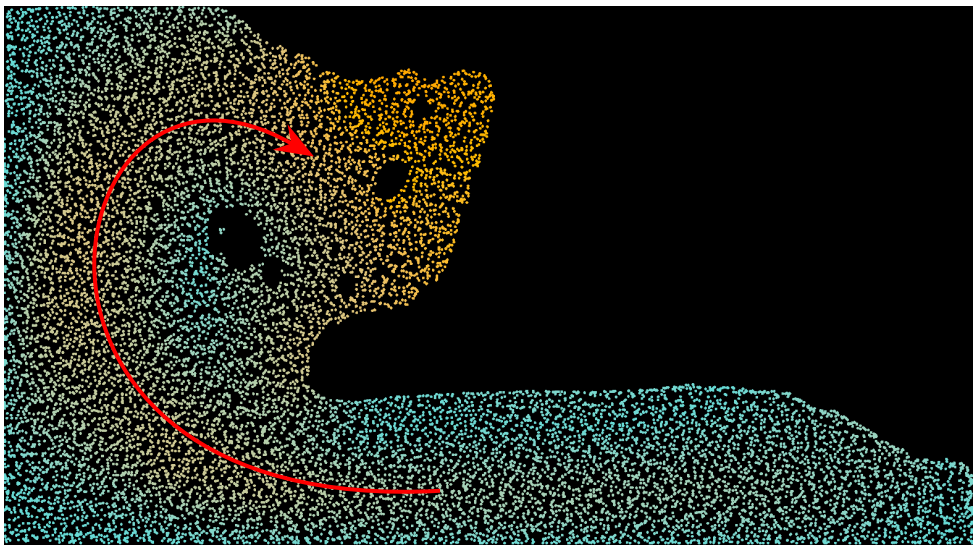


Figure 8: A demonstration of the RPIC method. We see that, whilst the 'gelatinous, clumpy' properties of PIC still exist (showing resistance to shear, particularly in the overturning wave) this greatly improves over PIC for rotational motion, as can be seen via the large vortex in red.

Applying this to our BiLerpPG operator conserves both linear and angular momentum, but at a cost of annihilating most *shear* fluid element deformations. This emerges directly from the approximation of rigid bodies [25], and can be observed in more detail in Figure 10.

3.3.3 Affine Approximations

We account for this lack of shear using the same techniques - we further dope our particles to handle translational modes, by approximating particle velocities as locally *affine* [23]. In the mathematical language we have constructed, this is achieved by replacing $\mathbf{R_p}$ with a *general* $\mathbb{R}^{2,2}$ matrix, rather than an anti-symmetric one (giving a new meaning to $\boldsymbol{\omega}_p^* \to \boldsymbol{\omega}_p^\dagger$ in formula (18), representing the local approximation of $\nabla \underline{u}_p$) [9]. Following from equation (22), we thus replace

$$\sum_p w_p^i \underline{u}_p^{\text{rigid}} \to \sum_p w_p^i \underline{u}_p^{\text{affine}}, \qquad \text{where} \quad \underline{u}_p^{\text{affine}} = \underline{u}_p + \omega_{\mathbf{p}}^\dagger(\underline{\chi}_i - \underline{x}_p). \tag{23}$$

Replacing anti-symmetric operators in (21) with generalised matrices gives $\omega_p \to \boldsymbol{\omega}_p^\dagger = \mathfrak{L}_p(\mathfrak{J}_p)^{-1}$, where

$$\mathfrak{J}_p = \sum_g w_p^g(\underline{\chi}_g - \underline{x}_p)(\underline{\chi}_g - \underline{x}_p)^T = \begin{bmatrix} (\underline{\Delta x}_x^{gp})^2 & \underline{\Delta x}_x^{gp}\underline{\Delta x}_y^{gp} \\ \underline{\Delta x}_x^{gp}\underline{\Delta x}_y^{gp} & (\underline{\Delta x}_y^{gp})^2 \end{bmatrix},$$

is analogous to the generalised inertia tensor as defined in equation (20), and

$$\mathfrak{L}_p = \sum_g w_p^g \underline{u}_g(\underline{\chi}_g - \underline{x}_p^{\text{old}})^T,$$

is akin to the generalised angular momentum loss tensor, generalised from equation (17) [9]. As $\underline{u}_g$ are the divergence-free grid velocities, this calculation must proceed the advection of particles (immediately after the BiLerpGP operator; hence the use of $\underline{x}_p^{\text{old}}$), for use in the next simulation step's BiLerpPG operator (during which we calculate $\mathfrak{J}_p$ using the advected particles). This naturally requires an initial condition for $\mathfrak{L}_p$ [23] (as BiLerpPG is the first operator we perform in each simulation step - see Section A.2), which comes from the particles' initial conditions. One can show that, in the specific case of linear interpolation (or indeed, bilinear), we have the identity

$$\begin{bmatrix} \frac{\partial w_p^i}{\partial \underline{x}_{p.x}} \\ \frac{\partial w_p^i}{\partial \underline{x}_{p.y}} \end{bmatrix} = \underline{\nabla w}_p^i = w_p^i \mathfrak{J}_p^{-1}(\underline{\chi}_i - \underline{x}_p^{\text{old}}), \tag{24}$$

where we label the components of $\begin{bmatrix} \underline{x}_{p.x} \\ \underline{x}_{p.y} \end{bmatrix} \equiv \underline{x}_p^{\text{old}}$ [9]. To calculate these partial derivatives, note that as the weight are calculated bilinearly, each component is separable. Hence,

$$\frac{\partial w_p^i}{\partial \underline{x}_{p.x}} = -\frac{\text{Lerp}'(\frac{\underline{\chi}_{p.x} - \underline{x}_{p.x}}{s})\,\text{Lerp}(\frac{\underline{\chi}_{p.y} - \underline{x}_{p.y}}{s})}{s}; \qquad \text{Lerp}(x) = \begin{cases} 1 - |x| & -1 < x < 1 \\ 0 & \text{else} \end{cases},$$

Using equation (24), we can re-define the interpolation operator (23) as

$$\sum_p w_p^i \underline{u}_p^{\text{affine}} = \sum_p \left(w_p^i \underline{u}_p + \mathfrak{L}_p \underline{\nabla w}_p^i \right), \tag{25}$$

meaning we never have to invert $\mathfrak{I}_p$ directly - we merely calculate the partial derivatives of w_p^i alongside the BiLerpGP operation, for use in the next simulation step's BiLerpPG.

As per usual, we require extra care for handling staggered MAC grid velocities. As with the standard bilinear operators, we solve this by segmenting our interpolation calculations into each grid face type (ie: x and y velocity components). Relating this to our new techniques, this involves creating new vectors $\{\underline{c}_p^x, \underline{c}_p^y\}$ for each face alongside BiLerpGP (which contains the column data of $\boldsymbol{\omega}_p^\dagger$) [9], and replacing formula (25) to get, for the specific case of $\underline{c}_p^x$,

$$\sum_p w_p^i \left(\underline{u}_{p.x} + \underline{c}_p^x \cdot (\underline{\chi}_i - \underline{x}_p) \right), \qquad \underline{c}_p^x = \sum_{1 \le i \le 4} \underline{\nabla w}_p^i (\underline{u}_{i.x}).$$

Finally, we must re-define the concept of total angular momentum in equations (16), for our 'affine' particles. Through our new conservation properties, we say that this is equal to the angular momentum of the state's *grid representation*, immediately after interpolation. Recall that any matrix can be decomposed into a symmetric and anti-symmetric matrix. Considering operator (18), we see that any purely rotational transformation can be represented by an anti-symmetric matrix. To extract the 'internal spin' of a velocity gradient $\nabla \underline{u}_p \approx \mathfrak{L}_p \mathfrak{I}_p^{-1}$, therefore, we extract its anti-symmetric component. In the case of APIC, the conservation of total angular momentum is elegantly tracked by said components of the affine matrix $\mathfrak{L}_p$ itself [9], and we therefore write this as a sum of the particles' 'orbital' angular momentum, and their internal spin:

$$L_P^{\text{affine}} = \sum_p (\underline{x}_p \times \underline{u}_p) + (\mathfrak{L}_{21} - \mathfrak{L}_{12}), \tag{26}$$

Pausing for a slight breath of relief, we observe APIC in practice. We relegate the proofs that these schemes preserve affine velocity fields, and linear & angular momentum, to Appendix C.

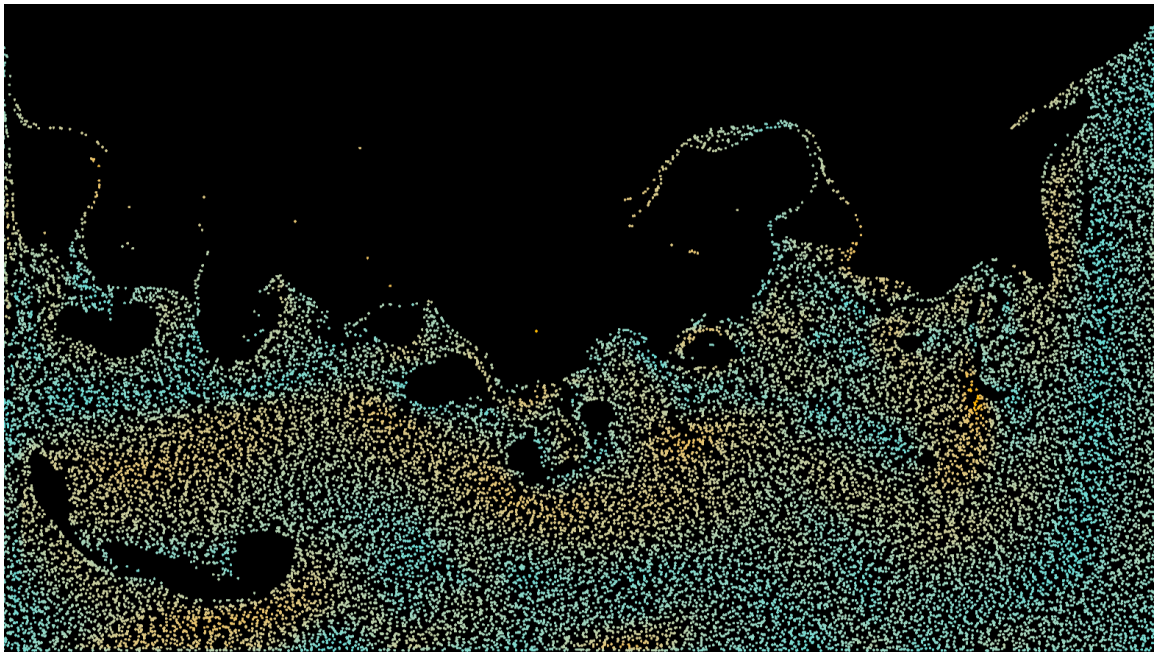


Figure 9: A real-time APIC simulation screenshot, with 25,000 particles. We see that, even for settings involving rough, high forces, many overturning waves, and extreme turbulence, APIC remains stable, and is very effective at preserving vortex motion, even on small scales (such as the small, numerous overturning waves shown in this image).

4 Results

4.1 Forces on Fluid Elements

Using the APIC angular momentum definitions in formula (26), we track the conservation properties of the various models we have derived thus far. First, however, we need confidence that are a suitable approximation for a general fluid flow - in particular, we wish to see emergence of vortices in the limit of high Reynolds number flow, and thus investigate the *shearing* of fluid elements, to see how malleable (and hence inertially-dominated) they are.

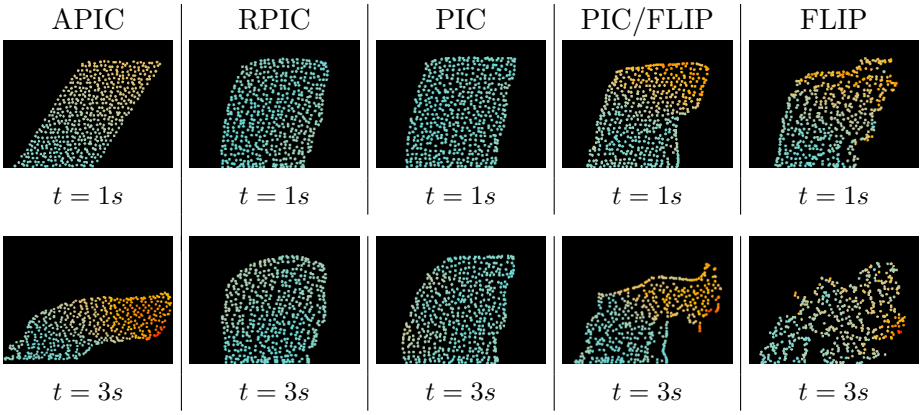


Figure 10: A shear test on a small fluid element, tested with various methods ($\alpha = 0.05$ for PIC/FLIP). Unlike PIC and RPIC, which are impervious to shearing, PIC/FLIP, FLIP, and APIC deform as expected. We also remark that $\alpha > 0$ is necessary for fluid elements to overturn, but that *any* presence of PIC prevents the bottom half from shearing effectively. This can be eliminated by using pure FLIP ($\alpha = 1$), at the cost of instability (particularly at the boundary of fluid elements). APIC has the best of both worlds, with discernable, linear shearing that cleanly overturns the entire length when the structure must rotate.

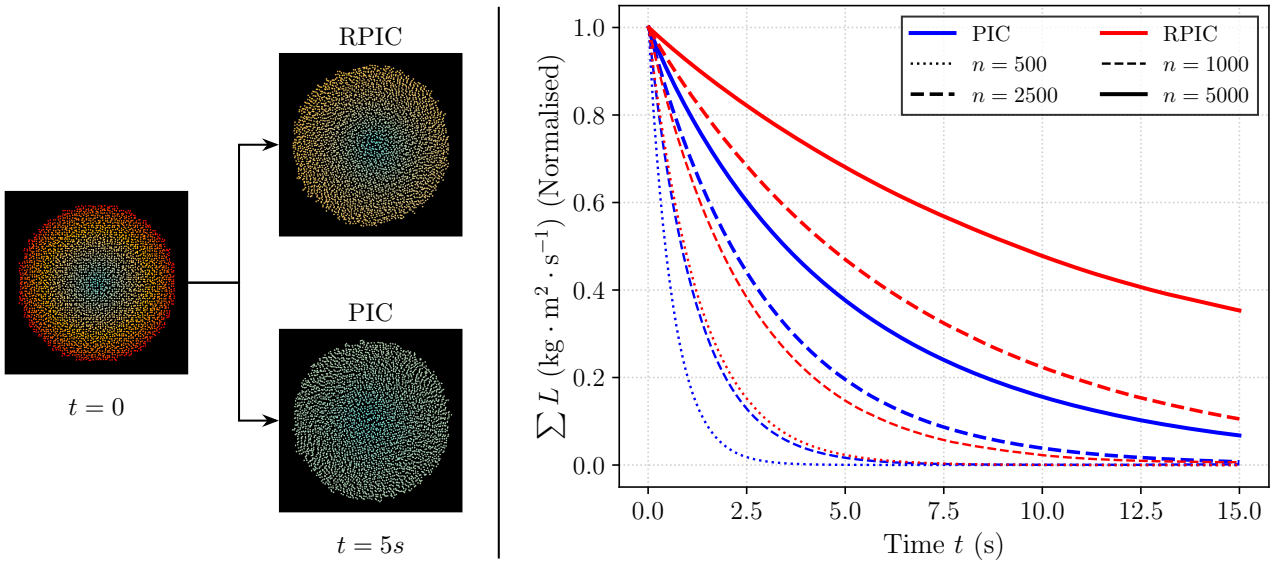
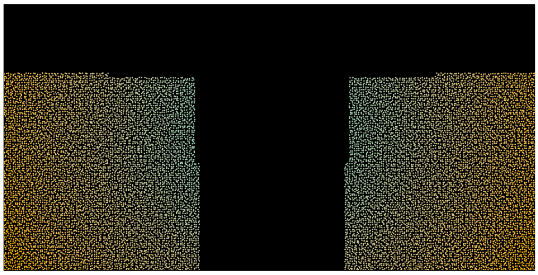


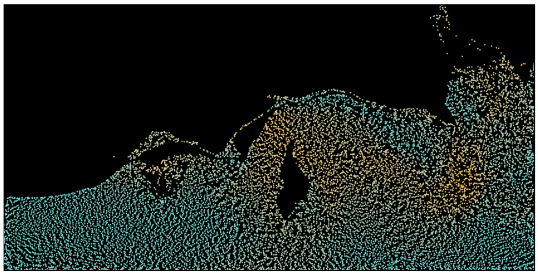
Figure 11: A comparison of the angular momentum conservation of RPIC and PIC, for an oscillating circle of n particles that are initialised with tangential velocity. We choose these models in particular due to a lack of shearing (see Figure 10 - energy modes are almost exclusively angular-based). We observe that RPIC preserves angular momentum significantly more than PIC; whilst both see more accurate results as n increases (presumably as the most angular momentum information is stored near the circle's edges), the *rate* of increase is much faster for RPIC than PIC. RPIC ($n = 5000$) test has the most linear-like decay in the suite.

4.2 Real-World Example: Overturning Waves

Figures 10 and 11 demonstrate that the local structure of fluid elements is resolved significantly more accurately by considering locally linear velocity fields. Particularly, the addition of symmetric velocity gradient components in APIC is essential for shear deformations, but only anti-symmetric components (such as those in RPIC's matrices) are required for angular momentum conservation in rotational deformations. We thus study the case of an overturning wave (which combines huge amounts of both shearing and rotation), to build confidence that APIC can effectively conserve all types of momentum, and is thus well-suited for general applications.



$t = -(0.77 \pm 0.2)s$



$t = 3.83s$ (APIC)

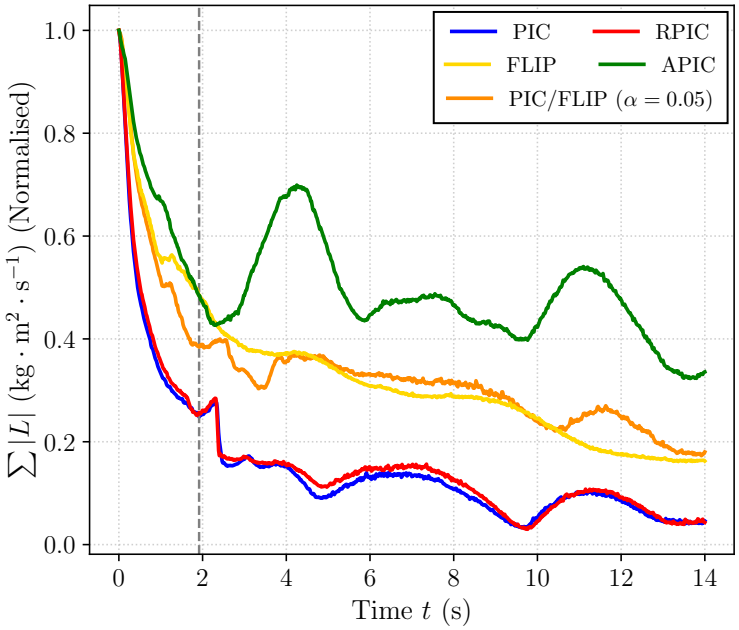


Figure 12: A numerical simulation demonstrating the angular momentum conservation properties, for an 'overturning wave' (averaged over 4 runs across a variety of models). We initialise 15,000 particles in two large squares of equal sizes, with small velocity normal to their vector from the simulation centre. This allows the right square to overturn the left. Timing begins once the two squares collide. Due to the high amount of shearing, PIC and RPIC have consistently poor performances, with substantial improvements upon addition of FLIP. However, as we increase α, the lack of noise reduction between neighbouring particles removes most global wave behaviour, resorting only to an (albeit moderate) amount of *local* vortex motion conservation. APIC, on the other hand, can capture and conserve local vortex motion, whilst retaining the global wave structure as seen in PIC and RPIC. The angular momentum conservations of APIC are especially prevalent in the initial wave overturn ($t \lesssim 2s$, in which results are more controlled), where we see comparable-to-better performance with the energy-conserving FLIP.

With these test in mind, we can have confidence that APIC improves massively over the PIC model (attaining the stability of $\alpha = 1$, whilst retaining the conservative properties of $\alpha = 0$). Figure 12 also reveals, however, that *no scheme* is fully impervious to numerical dissipation. Whilst this is partly due to the nature of the overturning wave problem, which produces thin sheets of thickness $\approx s$ (and thus cannot be accurately resolved by the grid) [26], the nature of using a finite-resolution grid will always result in an experimental contradiction, compared to the theoretical conservation properties (see Appendix C). Specifically, the numerical experiments reveal that low-frequency energy modes decay significantly slower than high-frequency modes, which locally *linear* velocities (such as those in RPIC and APIC) struggle to represent [27].

4.3 Taylor-Green Vortex: Analytical Case Study

The previous examples demonstrate that APIC outperforms the standard PIC model by a considerable margin, but have thus far failed to answer how applicable these methods are for large-scale, inviscid flow problems: we are clearly improving inside this 'black-box' of PIC methods, but are we converging to a ground truth? As a result, we turn towards investigating the developed methods' capabilities, under the canonical 2D 'Taylor-Green' vortex problem. Constructed in 1937 [28], the exact solution for this model (in Cartesian coordinates) can be summarised by,

$$\underline{u}(x, y, t) = \begin{pmatrix} \sin(\kappa_x x)\cos(\kappa_y y) \\ -\cos(\kappa_x x)\sin(\kappa_x y) \end{pmatrix} e^{-2\nu|\underline{\kappa}|^2 t}, \tag{27}$$

a separable equation into an initial vortical perturbation (with $\omega = 2\sin(x)\sin(y)$ [29]) and exponential damping. We thus require *periodic* boundary conditions, and as such $\kappa_x = \kappa_y = \frac{2\pi}{L}$ denotes the wavenumber for a square simulation of length L [30]. Transitioning away from our familiar Dirichlet boundary conditions requires a reconstruction of formulae (7) and (15), such that cells 'wrap' around the simulation's edges. We choose the fundamental $\kappa = 1$ in the analysis to come, which produces the vortices as depicted in Figure 13. To compare our regimes against the Taylor-Green vortex, we track the kinetic energy of the particles with initial condition given by expression (27), which follows,

$$E_k = E_0 e^{-4\nu\kappa^2 t}. \tag{28}$$

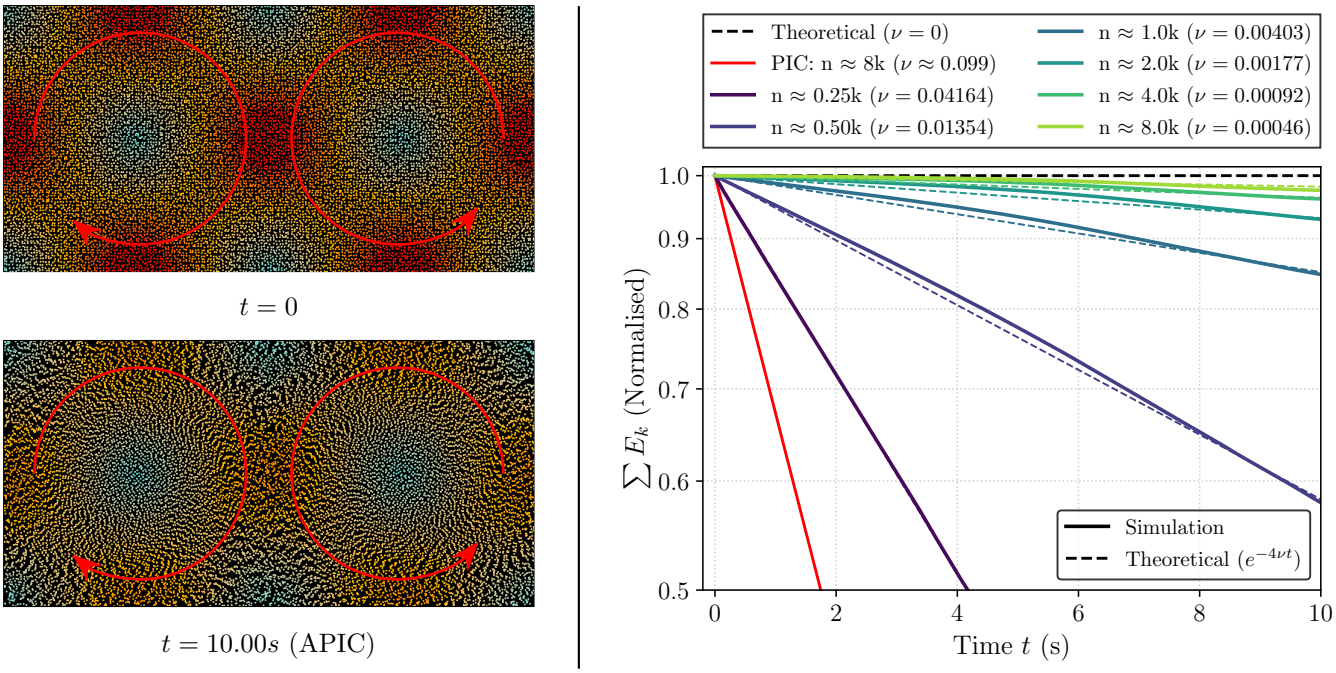


Figure 13: An analysis of the damping properties of APIC, for Taylor-Green vortices. The images on the left show a top-half cross-section of said vortices for a simulation of $32k$ particles, at various times, whilst the figure on the right plots normalised energy over time, for an increasing number of particles *per vortex*, n (recall that, via the $\kappa = 1$ mode, each simulation contains 4 such vortices). These simulated results are fitted to the theoretical trajectory given by solution (27), with free parameter ν. We also provide the analytical result for a truly inviscid flow, along with PIC performance, for metrics of comparison. Each test is repeated 5 times for consistency. Crucially, we observe that APIC converges nicely towards the inviscid solution, with a good 'performance to accuracy' balance emerging at $n \approx 3k$, after which convergence slows.

Fitting this model to various simulations of the Taylor-Green vortex, we can recover values of the kinematic viscosity ν for various APIC simulations, which are presented in Figure 13.

We make the following remarks based on the above figure. To begin with, despite being ran in a simulation with $32k$ particles, PIC struggles to conserve energy in these vortices, and performs on-par with $\approx 0.6k$ APIC particles ($n \approx 0.15k$), therefore strengthening our confidence in the results obtained in Figures 11 and 12. Further, for many APIC results we observe a characteristic 'settling' time, before which there is a slight upward deviation from the expected exponential decay of energy. This is a well-documented phenomenon [31] and, as this behaviour does not arise with the FLIP scheme, it is unlikely that we have probed the instability of APIC. Rather, this is likely due to incorrect initialisation of $\mathfrak{L}_p$ (this is set theoretically using $\nabla \underline{u}$ of formula (27), and does not take into account numerical errors in interpolation and projection). However, further analysis beyond this work is required to better characterise this behaviour.

Further numerical experiments were conducted with $n \approx 16k$ APIC particles, which recovered a theoretical value of $\nu = 3.7 \times 10^{-4}$ - a modest improvement over the best results seen in Figure 13. Making use of non-dimensional parameters (using $L = 98.4u$ to fit all our particles, where u is the standard Unity Game Engine unit), we derive an APIC Reynolds number of $Re = 5419.69$, whilst PIC yields $Re = 40.51$ for the same number of particles. These are upper bounds, with more realistic results (obtained using $n \approx 4k$) being $Re = 2213.68$ for APIC, and $Re = 10.27$ for PIC respectively. From this, we can conclude that *only* APIC (even at lower particle counts) has the ambition of being used for large-scale applications that model inviscid flows, and that, upon moving to higher particle counts, we appear to converge arbitrarily close to the inviscid solution (27). It is also worth mentioning that FLIP performs *incredibly* well in this test, due to its inherent energy conservation properties: for $n \approx 8k$, we yield a value of $\nu = 1.26 \times 10^{-4}$, corresponding to a Reynolds number approximately four times greater than that of APIC.

Without an established benchmark (due to a distinct lack of research in this area), one cannot be sure whether these values are reproducible. Nonetheless, this exploration serves as a novel validation for these constructed regimes, giving a metric of comparison with other methods in the field of CFD.

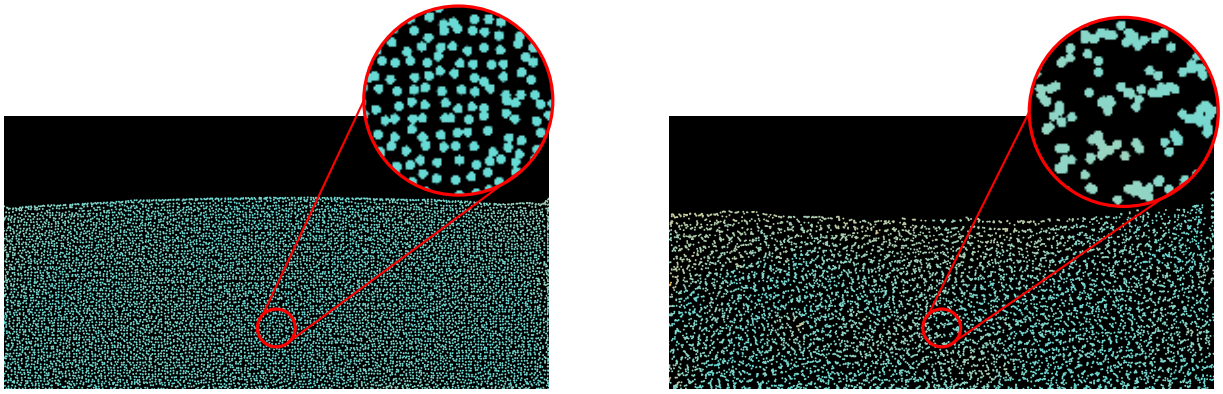
5 Conclusion

In this report, we have constructed a hybrid Particle-In-Cell method, that takes a 'best of both worlds' approach of Lagrangian advection and Eulerian projection techniques. We enforce this divergence-free property through the (Preconditioned) Conjugate Gradient Method, which required bijecting our MAC grid representation of the velocity field $\underline{u}$ (naturally emerging from Finite Difference Method techniques) to a sparse linear equation, consisting of the fluid's divergence and information of each cell's neighbours. The largest focus of conversation was spent on the main crux of these hybrid methods: the velocity transfers between the Lagrangian and Eulerian regimes. After building up the knowledge of how these interpolation transfer operators work, we carefully consider their inherent problems (namely, the loss of particle degrees of freedom), and the various entry points into solving these issues. The most promising of these ideas, that being to dope our particles with scalars or matrices representing angular momentum information, formed the premise of RPIC and APIC respectively. This required careful consideration to quantify the angular momentum and inertia of particle states, but rewards us with *significantly* stronger angular momentum conservation properties, compared to that of PIC. Whilst RPIC (like PIC) is ill-suited for shearing (despite having the sufficient conservation properties), doping with a full angular momentum matrix (rather than the anti-symmetric components in RPIC) restores these shearing modes, giving us the APIC method. This shows remarkable potential in terms of visual quality, local volume conservation, and the physical accuracy of turbulence (both in respect to PIC and other industry standards).

As computation of the velocity gradient matrices in APIC, and addition to the CGM solver, does not add significant performance overhead to the simulation ($\lesssim 10.1\%$, for the specific case study in Figure 12), and is incredibly well suited for parallelisation (due to our assumptions of particle momentum depending only on their *fixed* grid node neighbours, rather than their particle neighbours), whilst also retaining the stability of PIC over methods such as FLIP, it is the clear favourite in this particular suite.

Pure grid-based methods (such as *Stable Fluids*), while applicable for smooth, flow fluid motion, immensely struggle with angular information due to the sharp edges of cells, and apparent 'blurriness' of the pressure field over time [7]. Meanwhile, Lagrangian-based methods (such as SPH: *Smooth Particle Hydrodynamics*) are incredibly unstable, and struggle massively with volume conservation and particle clumping, making them a questionable solution to the Navier-Stokes equation [32]. These methods also call for the addition of external viscous forces, in order to well-approximate water, however the discretisation techniques required do not conserve angular momentum [33]. Naturally, there are many schemes which help SPH conserve momentum, however the framework they are built on is not as efficient as that of PIC: to calculate the vorticity for some particle p, one must consider the positions and velocities of all the particles in the rigid body χ. Quantifying this body becomes significantly easier in PIC methods (in Section 3.3.2, we consider χ to be the *MAC grid* components that encompass p); SPH methods require the application of a local smoothing kernel about p [33], typically via expensive nearest neighbour searches [34].

Unfortunately, APIC (which, like PIC, is still based on the assumption of material continuity) can suffer too from the problem of particle clumping, and therefore cannot effectively handle particle separation [35]. Resultantly, over time particles will begin to merge together and behave



(a) $t = -0.5$s (b) $t = 146$s

Figure 14: A small-scale simulation demonstrating the inherent merging of particles in the PIC method. At $t = \{0, 10, 20, ..., 60\}s$ we induce an impulse of artificial, strong, uniform forces throughout the fluid, then allow it to settle. We see that, although the fluid's volume is stable, many particles have joined to form 'clumps', which amplifies the numerical dissipation and instability present in the simulation.

as one, reducing the degrees of freedom of the particle representation [36]. Some techniques are available to account for this, however (albeit at a performance cost), such as adding an intrinsic repulsion force to pairs of particles, whenever they cross some distance threshold. Implementing this effectively requires the use of the Eulerian grid to check only the particles in some neighbourhood.

Nonetheless, this report (alongside the accompanying simulation codebase) hopefully provides the sufficient groundwork for extending these methods to new and exciting domains. For instance, by revisiting the assumption of treating air as a vacuum in Section 2.2.1, there are several considerations for future work, such as modelling wave generations by surface winds, or persistent bubbles inside our fluid [36] (both of which make use of the presence and compressibility of air). Another exciting stepping point for future interest is the consideration of surface tension in our fluid: one particular curiosity is that, in our transition from PIC/FLIP to APIC, we seem to lose some inherent clumping of particles near the boundary of fluid elements (observe Figure 10), presumably from reducing numerical instability near boundaries. Hence, it is perhaps *more* natural to use FLIP when introducing surface tension effects; indeed, this is the case for many new schemes, such as Yi-Lun Chen [37].

Despite these (albeit surmountable) limitations, it is clear that Affine Particle-In-Cell methods have (and will continue to develop) an incredibly promising legacy in the world of large-scale Computational Fluid Dynamics. Having only been developed recently in 2015 [9], there is a world of domains and regimes yet to be explored by these emergent algorithms, and one can therefore be positive that new and exciting research will be undoubtedly sparked by the robust solutions that APIC provides.

A Implementation & Optimisations of PIC

A.1 Optimisation Techniques

Whilst the Conjugate Gradient Method is mathematically elegant, blindly following Section 2.2.3 algorithmically, using the standard 2D-array programming structure, produces a variety of frustrating effects. Put bluntly, we need $||\underline{r}_\infty||$ to be 'as small as possible' for 'accurate' results, and so naturally there comes a strict balance between speed of convergence, and physical relevance of results. A primary goal is for our simulation to run in real-time, and thus be user-interactive - we therefore strive to recover as much physical accuracy we can get within this (albeit optimistic) window, in which we can break our efforts into two categories:

- Improving the efficiency of the CGM, to allow $\underline{\mathfrak{p}}_n$ to converge within some threshold of $\underline{\mathfrak{p}}_\infty$ in fewer iterations, meaning we can perform more simulation steps per second. Such methods involve Preconditioning (which we met in Section 2.2.4), and CSR Matrices.
- Improving the physical accuracy of a fluid that has a fixed CGM residue of $||\underline{r}||_\infty$, meaning we can perform fewer iterations and still retain realistic results. Such methods involve Volume Control and Particle Separation (see Figure 14).

A.1.1 Compressed Sparse-Row (CSR) Matrices

As we saw in Section 2.2.4, we use *Incomplete Cholesky Factorisation* to decompose our matrix $\mathbf{A} \approx \mathbf{M} = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ and $\mathbf{L}^T$ are sparse lower & upper triangular matrices respectively. By this sparsity, we can *massively* reduce the memory complexity of the CGM by storing $\mathbf{L}$ in the CSR format - to get a feel of the importance of this, note that we require a minimum of ≈ 60 simulation steps per second for 'smooth' graphics, each of which requiring a full reconstruction and disposing of $\mathbf{A} \in \mathbb{R}^{n_t, n_t}$. The CSR format operates by maintaining three arrays:

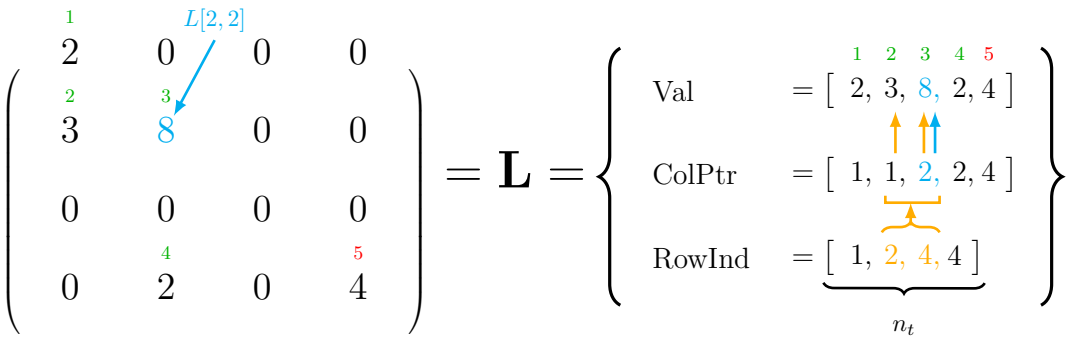


Figure 15: Representation of a matrix $\mathbf{L}$ in CSR form. Here, m is the number of non-zero entries, $= 5$ in this case. We also see a visualisation of equation (29) for $\mathbf{L}[2,2] = 8$, by indexing through all non-zero entries in row 2 of the matrix, stopping only when we have a column match.

The Val $\in \mathbb{R}^m$ array holds all non-zero matrix elements, with column values as in ColPtr $\in \mathbb{R}^m$ (eg: if $\mathbf{L}[2,1] = 3$ and Val$[i] = 3$ points to this value for some i, then ColPtr$[i] = 1$). RowInd $\in \mathbb{R}^{n_t}$ contains the total number of all non-zero matrix elements reached by the start of that row, $+1$. For example, in Figure 15, RowInd$[2] = 2$, meaning that by the start of row 2, we have counted 1 non-zero entry, and so the first non-zero entry in row 2 has index $1 + 1 = 2$ in the Val array. Put another way, each neighbouring pair of RowInd values represents an index bound in Val, holding all the non-zero entries contained in that row of $\mathbf{L}$. (Note that we often represent RowInd $\in \mathbb{R}^{n_t+1}$ with RowInd$[n_t + 1] = m$, in order to make our equations neater, such as in

equations (29) and (30).) We see that this implementation thrives under vector multiplication, by constructing the simple *GetValue* and *VectorProduct* operators respectively:

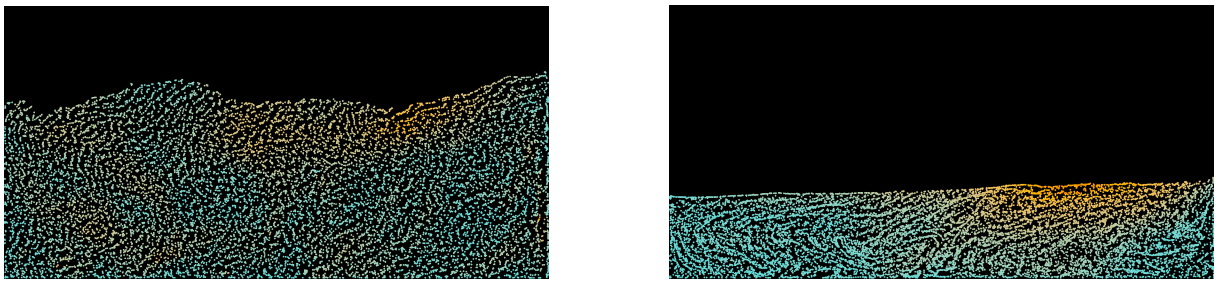
$$\mathbf{L}[i,j] = \sum_{k=\text{RowInd}[i]}^{\text{RowInd}[i+1]-1} \text{Val}[k]\ \delta_{\text{ColPtr}[i]j}\ ; \qquad \delta_{\text{ColPtr}[i]j} = \begin{cases} 1 & \text{ColPtr}[i] = j \\ 0 & \text{else} \end{cases}. \tag{29}$$

$$(\mathbf{L}\underline{v})_i = \sum_{k=\text{RowInd}[i]}^{\text{RowInd}[i+1]-1} \text{Val}[k]\ \underline{v}_{\text{ColPtr}[k]}. \tag{30}$$

However, as discussed previously in Section 2.2.4, we equally need to handle operations involving $\mathbf{L}^T$, which requires an implicit transfer to the 'Compressed Sparse-Column' format. The specifics of this, along with other relating algorithms, are significant enough a deviation to neglect in this particular essay. However novel algorithms have been developed in the code-base to handle this.

A.1.2 Volume Control

Even with these efficiency improvements, we are still masking the fundamental problem that our pCGM algorithm is merely an *approximation* method, and thus given any error threshold, there will always be some amount of divergence that is uncorrected for (which will then be heightened in bilinear interpolation). This inherent flaw results in the volume of our fluid not being preserved over time-steps, and naturally favours a *decrease* in volume [3].



(a) $t = 5.2$s (b) $t = 31.0$s

Figure 16: A small-scale simulation demonstrating the inherent volume loss in the Conjugate Gradient Method, where the error threshold has been exaggerated for effect.

To help control this [38], we add an additional term to our divergence $\underline{d}$ in formula (5) (representing the volume error in each cell) to form a *desired* divergence $\underline{c}$, which can then be used in place of $\underline{d}$ in the linear equation (8). This is calculated as such:

$$c(a,b) = d(a,b) + \kappa\left(\frac{\rho_0}{\rho(a,b)} - 1\right) \quad \text{with} \quad \rho_0 = \frac{\sum_{i=1}^{n_t}\rho(f(i))}{n_t},$$

$$\rho(a,b) := \text{Number of fluid particles inside cell [a,b]}.$$

κ here is called the *controller gain* (a stiffness constant or 'surface tension' effect, of sorts) - I vary $\kappa \in [0.5, 1.0]$ in my simulation for a good mix of stability against performance. For example, APIC conservation properties (in Figure 13) perform significantly better with κ small, however surface tension effects are essential for modelling the shearing of fluid elements as in Figure 10. Many other (more extreme) schemes for choosing κ are available - an example of this is one recommended by B. Kim [39]: $\kappa = \ln(10)/25\Delta t$, which can be shown to correct 90% of

the volume error in 25 simulation steps.

A.2 PIC Algorithmic Outline

We formulate a step-by-step plan for implementing the Particle-In-Cell Method [3, 15], from the analysis of Section 2.

1. Initialise our grid, labelling solid cells as required.
2. Initialise particles inside the grid, with regular spacings (typically 4 particles per cell).
3. Label all (non-solid) cells with particles as 'fluid', and all others with 'air'.
4. Apply the BiLerpPG operator to transfer to the grid representation.
5. Apply external forces to the grid (such as gravity).
6. Enforce Dirichlet boundary conditions: $\mathfrak{u} \cdot \underline{n} = 0$, for each cell that neighbours a solid cell, with intersection normal $\underline{n}$. Note that we neglect the stronger *no-slip* boundary condition $\mathfrak{u} = 0$, due to our model of inviscid fluids [3].
7. Create a linked list of fluid cells, and use this to construct the Coefficient Matrix (7).
8. Compute the divergence for each fluid cell, creating the linear equation $\mathbf{A}\mathfrak{p} = \mathfrak{b}$.
9. Solve the above linear system using the (Preconditioned) Conjugate Gradient Method, giving a value for the pseudo-pressure of each cell.
10. Apply the discrete gradient operator to these pressure values (mixed with Neumann BCs) to produce a divergence-free grid velocity field.
11. Apply the BiLerpGP operator to transfer to the particle representation.
12. Advect the particles using Semi-Euler Integration, or (better yet) RK2 / RK3.
13. Correct for Dirichlet BC numerical errors (typically produced via advection) by pushing particles out of solid cells, if needed. In my simulation, we do this by finding the point of fluid-solid intersection on the particle's path, then placing the particle at this point, nudged outwards slightly to enforce a 'boundary layer' effect.
14. Repeat from Step 3.

One final thing we haven't covered is the size of the time-step dt between simulation steps. Naturally, the smaller dt is, the more 'stable and realistic' our model will be, however there is a crucial upper bound to place on this: the *CFL condition* [15]. This states that hybrid schemes such as PIC can *never be stable* if particles are allowed to 'jump over' a cell during one simulation step. This is enforced by setting $dt = \min(dt, s/v_{\max})$ after each simulation step, where $v_{\max} = \max(|v_p| : \text{particles } p)$. Ideally, we should fall short of $s/v_{\max}$ by some factor: as particle advection techniques typically use a form of Runge-Kutta (where we split dt into smaller chunks) more stability can be achieved (at the cost of speed) by enforcing a stricter CFL condition after each simulation sub-step. I've personally found a good balance with RK2, with a 2x quarter-cell cut-off for each sub-step.

B Proof of Helmholtz Decomposition (Theorem 2.3)

We begin by introducing the concept of a *Dirac-Delta*. This concept has its roots in many streams, such as Fourier Transforms & Convolutions (ie: the inverse of the function $f(x) = 1$), the general solution to the Heat Equation (ie: $S(x,t) = e^{-\frac{x^2}{4\kappa t}}/\sqrt{4\pi\kappa t}$, and (the source of this section's discussion) Green's Functions, and thus has many formal definitions. In a very informal sense, we define a *Dirac-Delta* to be the derivative of the Heaviside step function $H(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$, or rather $\delta(x) = \begin{cases} \infty & \text{if } x = 0 \\ 0 & else \end{cases}$. As such, in most cases this should be treated less as a 'function' of sorts, but more as a *distribution*. Whatever its specific form, the 'defining property' of a Dirac-Delta is to solve, for some continuous function $f : \mathbb{R}^n \to \mathbb{R}^m$:

$$f(x) = \int_V f\left(x'\right) \delta\left(x - x'\right) dV' \tag{31}$$

Lemma B.1. *The function*

$$\delta^2\left(x\right) := -\frac{1}{4\pi}\nabla^2\left(\frac{1}{||x||}\right) \tag{32}$$

is a well-defined Dirac-Delta Function for $u \in C^1(V, \mathbb{R}^2)$ and $V \subset \mathbb{R}^2$ open, in the sense of (31).

We will take this lemma for granted, and use it to prove the main theorem of this section.

Proof of Theorem 2.3 (Helmholtz Decomposition in $\mathbb{R}^2$) [13].

Subbing u and δ^2 into equation (31) gives,

$$u(x) = -\frac{1}{4\pi}\int_V \frac{\nabla^2 u(x')}{||x - x'||} dV'.$$

Using the identity $\nabla^2 u = \nabla(\nabla \cdot u) \; - \; \nabla \times (\nabla \times u)$, we get

$$u(x) = \underbrace{\nabla\overbrace{\left(\nabla \cdot \int_V \frac{-u(x')}{4\pi||x - x'||} dV'\right)}^{\phi(x)}}_{\mathcal{C}(x)} + \underbrace{\nabla \times \overbrace{\left(\nabla \times \int_V \frac{u(x')}{4\pi||x - x'||} dV'\right)}^{\mathcal{A}(x)}}_{\mathcal{D}(x)}.$$

We now show that $\mathcal{C}$ and $\mathcal{D}$ have the desired properties of $\nabla \times \mathcal{C} = 0$ and $\nabla \cdot \mathcal{D} = 0$. Note that taking the curl of $\mathcal{C}$ gives,

$$\nabla \times \nabla\phi = \frac{\partial^2\phi}{\partial x \partial y} - \frac{\partial^2\phi}{\partial y \partial x} = 0,$$

for any $\phi \in C^1(V, \mathbb{R})$ (as $u \in C^2(V, \mathbb{R}^2)$) - we conclude that ϕ is a *Scalar Potential*, and $\nabla \times \mathcal{C} = 0$. Similarly, taking the divergence of $\mathcal{D}$ gives

$$\nabla \cdot (\nabla \times \mathcal{A}) = \frac{\partial^2\mathcal{A}}{\partial x \partial y} - \frac{\partial^2\mathcal{A}}{\partial y \partial x} = 0,$$

for any $\mathcal{A} \in C^1(V, \mathbb{R}^2)$ (as $u \in C^2(V, \mathbb{R}^2)$) - $\mathcal{A}$ is a *Vector Potential*, and $\nabla \cdot \mathcal{D} = 0$. □

C Key Proofs & Mathematical Rigour of Affine Transfers

Note that this section relies heavily on the work by C. Jiang [40]. We begin by summarising the APIC velocity transfer schemes:

From particle to grid representations,

$$m_g \underline{u}_g = \sum_p w_p^g (\underline{u}_p + \mathfrak{L}_p \mathfrak{I}_p^{-1} (\underline{\chi}_g - \underline{x}_p)), \tag{33a}$$

$$\mathfrak{I}_p = \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p)(\underline{\chi}_g - \underline{x}_p)^T, \tag{33b}$$

$$m_g = \sum_p w_p^i. \tag{33c}$$

From grid to particle representations,

$$\underline{u}_p = \sum_g w_p^g \underline{u}_g, \tag{34a}$$

$$\mathfrak{L}_p = \sum_g w_p^g \underline{u}_g (\underline{\chi}_g - \underline{x}_p)^T. \tag{34b}$$

The total affine angular momentum on particles,

$$L_P = \sum_p (\underline{x}_p \times \underline{u}_p) + (\mathfrak{L}_p^T : \boldsymbol{\epsilon}), \tag{35a}$$

$$\mathbf{A} : \boldsymbol{\epsilon} = \mathbf{A}_{12} - \mathbf{A}_{21}. \tag{35b}$$

We also state the useful identities, for use in the below proofs.

$$\sum_g w_p^g = 1 \tag{36a}$$

$$\sum_g w_p^g \underline{\chi}_g = \underline{x}_p \tag{36b}$$

$$\underline{a} \times \mathbf{M}\underline{b} = (\mathbf{M}\underline{b}\underline{a}^T)^T : \boldsymbol{\epsilon} \tag{36c}$$

C.1 Preservation of Affine Velocity Fields

Proposition C.1. *Consider the act of transferring from the grid $\underline{\tilde{u}}_g$ to particle state $(\underline{u}_p, \mathfrak{L}_p)$, then back to the grid $\underline{u}_g$. If $\underline{\tilde{u}}_g$ represents an affine velocity field (in the sense of Definition 3.1: $\underline{\tilde{u}}_g = \underline{v} + \mathbf{C}\underline{\chi}_g$), then $\underline{u}_g = \underline{\tilde{u}}_g$ in the limit of $\Delta t = 0$ (used to specifically extract the interpolation operators from the simulation).*

Proof. As $\Delta t = 0$, particle positions remain constant across the transfer, so $\mathfrak{I}_p$ and m_g remain constant also. We first perform BiLerpGP on some particle p using formula (34):

$$\underline{u}_p \;=\; \sum_g w_p^g \underline{\tilde{u}}_g \sum_g w_p^g (\underline{v} + \mathbf{C}\underline{\chi}_g) \;=\; \underline{v} \sum_g w_p^g + \mathbf{C} \sum_g w_p^g \underline{\chi}_g \;=\; \underline{v} + \mathbf{C}\underline{x}_p.$$

And construct $\mathfrak{L}_p$ via:

$$\mathfrak{L}_p = \sum_g w_p^g \tilde{\underline{u}}_g (\underline{\chi}_g - \underline{x}_p)^T = \sum_g w_p^g (\underline{v} + \mathbf{C}\underline{\chi}_g)(\underline{\chi}_g - \underline{x}_p)^T = \underline{v} \left(\sum_g w_p^g (\underline{\chi}_g - \underline{x}_p) \right)^T + \mathbf{C} \sum_g w_p^g \underline{\chi}_g (\underline{\chi}_g - \underline{x}_p)^T.$$

The first term vanishes from formula (36b); writing $\underline{\chi}_g = (\underline{\chi}_g - \underline{x}_p) + \underline{x}_p$ in the second term and using formula (33b) gives

$$\mathfrak{L}_p = \mathbf{C} \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p)(\underline{\chi}_g - \underline{x}_p)^T + \mathbf{C}\underline{x}_p \left(\sum_g w_p^g (\underline{\chi}_g - \underline{x}_p) \right)^T = \mathbf{C}\mathfrak{J}_p.$$

We then perform BiLerpPG on p using formula (33):

$$\begin{aligned} m_g \underline{u}_g &= \sum_p w_p^g \left(\underline{u}_p + \mathfrak{L}_p \mathfrak{J}_p^{-1} (\underline{\chi}_g - \underline{x}_p) \right) \\ &= \sum_p w_p^g \left((\underline{v} + \mathbf{C}\underline{x}_p) + (\mathbf{C}\mathfrak{J}_p) \mathfrak{J}_p^{-1} (\underline{\chi}_g - \underline{x}_p) \right) \\ &= \sum_p w_p^g \left(\underline{v} + \mathbf{C}\underline{x}_p + \mathbf{C}(\underline{\chi}_g - \underline{x}_p) \right) \\ &= \sum_p w_p^g (\underline{v} + \mathbf{C}\underline{\chi}_g) = m_g (\underline{v} + \mathbf{C}\underline{\chi}_g). \end{aligned}$$

Hence, $\underline{u}_g = \underline{v} + \mathbf{C}\underline{\chi}_g = \tilde{\underline{u}}_g$. □

C.2 Conservation of Linear Momentum

Proposition C.2. *Total linear momentum is conserved in the Affine* BiLerpPG *operator.*

Proof. Using formula (33),

$$\begin{aligned} \sum_g m_g \underline{u}_g =: \mathtt{p}_G &= \sum_g \sum_p w_p^g \left(\underline{u}_p + \mathfrak{L}_p \mathfrak{J}_p^{-1} (\underline{\chi}_g - \underline{x}_p) \right) \\ &= \sum_p \left(\sum_g w_p^g \right) \underline{u}_p + \sum_p \mathfrak{L}_p \mathfrak{J}_p^{-1} \left(\sum_g w_p^g (\underline{\chi}_g - \underline{x}_p) \right). \end{aligned}$$

Using formulae (36a) and (36b), this reduces to

$$\mathtt{p}_G = \sum_p \underline{u}_p =: \mathtt{p}_P$$

□

Remark C.3. *By our previous analysis in Section 3.3, this property immediately applies to* BiLerpGP*. We can also apply the same logic to the conservation of angular momentum.*

C.3 Conservation of Angular Momentum

Proposition C.4. *Angular momentum is conserved during the APIC transfer from particles to the grid.*

Proof. Substituting equation (16) into formula (33) gives

$$L_G = \sum_g \underline{\chi}_g \times m_g \underline{u}_g = \sum_g \underline{\chi}_g \times \left(\sum_p w_p^g \left(\underline{u}_p + \mathfrak{L}_p \mathfrak{I}_p^{-1} (\underline{\chi}_g - \underline{x}_p) \right) \right)$$
$$= \sum_p \left(\sum_g w_p^g \underline{\chi}_g \right) \times \underline{u}_p + \sum_p \sum_g w_p^g \underline{\chi}_g \times \left(\mathfrak{L}_p \mathfrak{I}_p^{-1} (\underline{\chi}_g - \underline{x}_p) \right).$$

The first term simplifies to $\sum_p \underline{x}_p \times \underline{u}_p$ by formula (36b). For the second term, we decompose $\underline{\chi}_g = (\underline{\chi}_g - \underline{x}_p) + \underline{x}_p$, and use the distributivity of the cross product to get,

$$\sum_p \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p) \times \left(\mathfrak{L}_p \mathfrak{I}_p^{-1} (\underline{\chi}_g - \underline{x}_p) \right)$$
$$+ \sum_p \underline{x}_p \times \left(\mathfrak{L}_p \mathfrak{I}_p^{-1} \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p) \right), \quad \text{0 by (36b)}$$

Using the identity (36c), we conclude that

$$L_G = \sum_p \underline{x}_p \times \underline{u}_p + \left(\mathfrak{L}_p \mathfrak{I}_p^{-1} \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p)(\underline{\chi}_g - \underline{x}_p)^T \right)^T : \epsilon$$
$$= \sum_p \underline{x}_p \times \underline{u}_p + \left(\mathfrak{L}_p \mathfrak{I}_p^{-1} \mathfrak{I}_p \right)^T : \epsilon$$
$$= \sum_p (\underline{x}_p \times \underline{u}_p) + (\mathfrak{L}_p^T : \epsilon) \;=\; L_P. \qquad \square$$

Proposition C.5. *Angular momentum is conserved during the APIC transfer from the grid to particles.*

Proof. Again, using the identity (36c), we get

$$\sum_p \mathfrak{L}_p^T : \epsilon = \sum_p \left(\sum_g w_p^g \tilde{\underline{u}}_g (\underline{\chi}_g - \underline{x}_p^{\text{old}})^T \right)^T : \epsilon$$
$$= \sum_p \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p^{\text{old}}) \times \tilde{\underline{u}}_g,$$

for particle positions $\underline{x}_p^{\text{old}}$ before advection. Hence, substituting into formula (35) before the next BiLerpPG operation (to produce particle positions $\underline{x}_p^{\text{new}}$ via advection) gives:

$$L_P = \sum_p \underline{x}_p^{\text{new}} \times \underline{u}_p + \sum_p \sum_g w_p^g \underline{\chi}_g \times \tilde{\underline{u}}_g - \sum_p \left(\sum_g w_p^g \right) \underline{x}_p^{\text{old}} \times \tilde{\underline{u}}_g$$
$$= \sum_p (\underline{x}_p^{\text{new}} - \underline{x}_p^{\text{old}}) \times \underline{u}_p + \sum_g \left(\sum_p w_p^g \right) \underline{\chi}_g \times \tilde{\underline{u}}_g$$
$$= \sum_p (\Delta t \underline{u}_p) \times \underline{u}_p + L_G \;=\; L_G,$$

where the last line is given from the simple semi-Euler advection scheme between time-steps. $\square$

References

[1] Gregory L Eyink. Josephson-anderson relation and the classical d'alembert paradox. *Physical Review X*, 11(3):031054, 2021.

[2] Vortex Bladeless: A Wind Generator without Blades. https://vortexbladeless.com, 2026.

[3] R. Bridson. *Fluid Simulation for Computer Graphics*. Taylor & Francis, 2008.

[4] Disney Animation. Moana: Crashing waves. https://media.disneyanimation.com/uploads/production/publication_asset/164/asset/Moana_Crashing_Waves.pdf, 2016.

[5] Shahriyar Shahrabi. Gentle introduction to fluid simulation for programmers and technical artists. Medium, 2017.

[6] Martha W Evans, Francis H Harlow, and Eleazer Bromberg. The particle-in-cell method for hydrodynamic calculations. Technical report, 1957.

[7] Jos Stam. Stable fluids. In *Proceedings of the 26th annual conference on Computer graphics and interactive techniques (SIGGRAPH)*, pages 121–128, 1999.

[8] J.U. Brackbill, D.B. Kothe, and H.M. Ruppel. Flip: A low-dissipation, particle-in-cell method for fluid flow. *Computer Physics Communications*, 48(1):25–38, 1988.

[9] Chenfanfu Jiang, Craig Schroeder, Andrew Selle, Joseph Teran, and Alexey Stomakhin. The affine particle-in-cell method. *ACM Trans. Graph.*, 34(4), July 2015.

[10] COMSOL. The finite element method (FEM). https://www.comsol.com/multiphysics/finite-element-method, 2026.

[11] Colin Braley and Adrian Sandu. Fluid simulation for computer graphics: A tutorial in grid based and particle based methods. *Virginia Tech, Blacksburg*, 3, 2010.

[12] Richard Fitzpatrick. The Helmholtz theorem. University of Texas at Austin, Electromagnetism Lecture Notes, 2007.

[13] Christopher S. Baird. Helmholtz decomposition of vector fields. University of Massachusetts Lowell, 2013.

[14] Felix Schulze. MA263 - multivariable analysis. Lecture Notes, University of Warwick, 2025.

[15] Dan Englesson. Real-time FLIP fluid simulation on the GPU. Master's thesis, Lund University, 2016.

[16] Sheldon Axler. *Linear Algebra Done Right*. Springer, 3rd edition, 2015.

[17] Jonathan Richard Shewchuk. An introduction to the conjugate gradient method without the agonizing pain. Technical report, Carnegie Mellon University, 1994.

[18] Chih-Jen Lin and Jorge J Moré. Incomplete cholesky factorizations with limited memory. *SIAM Journal on Scientific computing*, 21(1):24–45, 1999.

[19] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

[20] Ziyin Qu, Minchen Li, Fernando De Goes, and Chenfanfu Jiang. The power particle-in-cell method. *ACM Transactions on Graphics*, 41(4), 2022.

[21] Ounan Ding, Tamar Shinar, and Craig Schroeder. Affine particle in cell method for mac grids and fluid simulation. *Journal of Computational Physics*, 408:109311, 2020.

[22] Jeremiah U Brackbill and Hans M Ruppel. Flip: A method for adaptively zoned, particle-in-cell calculations of fluid flows in two dimensions. *Journal of Computational physics*, 65(2):314–343, 1986.

[23] Michael Tupek, Jacob Koester, and Matthew Mosby. A momentum preserving frictional contact algorithm based on affine particle-in-cell grid transfers. *arXiv preprint arXiv:2108.02259*, 2021.

[24] Herbert Goldstein, Charles Poole, and John Safko. *Classical Mechanics*. Addison Wesley, 3rd edition, 2002.

[25] Chenfanfu Jiang, Craig Schroeder, Andrew Selle, Joseph Teran, and Alexey Stomakhin. The affine particle-in-cell method (video supplement). https://www.youtube.com/watch?v=jPG5H5ZoL5Y, 2015.

[26] Jong-Hyun Kim, Wook Kim, Young Bin Kim, and Jung Lee. Visual simulation of detailed turbulent water by preserving the thin sheets of fluid. *Symmetry*, 10(10):502, 2018.

[27] Chuyuan Fu, Qi Guo, Theodore Gast, Chenfanfu Jiang, and Joseph Teran. A polynomial particle-in-cell method. *ACM Transactions on Graphics (TOG)*, 36(6):1–12, 2017.

[28] Geoffrey Ingram Taylor and Albert Edward Green. Mechanism of the production of small eddies from large ones. *Proceedings of the Royal Society of London. Series A-Mathematical and Physical Sciences*, 158(895):499–521, 1937.

[29] Yuval Dagan. Analytical solutions for particle dispersion in taylor–green vortex flows: Y. dagan. *Theoretical and Computational Fluid Dynamics*, 39(1):14, 2025.

[30] Dimitris Drikakis, Christer Fureby, Fernando F Grinstein, and David Youngs. Simulation of transition and turbulence decay in the taylor–green vortex. *Journal of Turbulence*, (8):N20, 2007.

[31] Ben D Fudge, Radu Cimpeanu, and Alfonso A Castrejón-Pita. Drop impact onto immiscible liquid films floating on pools. *Scientific Reports*, 14(1):13671, 2024.

[32] Yumeng He, Hsin Li, Yuchen Chen, and Xu Chen. Incompressible fluid simulation: A comparison. Technical report, University of Southern California, 2025.

[33] XY Hu and NA Adams. Angular-momentum conservative smoothed particle dynamics for incompressible viscous flows. *Physics of Fluids*, 18(10), 2006.

[34] Markus Ihmsen, Nadir Akinci, Markus Becker, and Matthias Teschner. A parallel sph implementation on multi-core cpus. In *Computer Graphics Forum*, volume 30, pages 99–112. Wiley Online Library, 2011.

[35] Chenhui Wang, Jianyang Zhang, Chen Li, and Changbo Wang. Dc-apic: A decomposed compatible affine particle in cell transfer scheme for non-sticky solid–fluid interactions in mpm. *Graphical Models*, 139:101269, 2025.

[36] Matthias Müller. FLIP fluid simulation. https://matthias-research.github.io/pages/tenMinutePhysics/18-flip.pdf, 2021.

[37] Yi-Lu Chen, Jonathan Meier, Barbara Solenthaler, and Vinicius C Azevedo. An extended cut-cell method for sub-grid liquids tracking with surface tension. *ACM Transactions on Graphics (TOG)*, 39(6):1–13, 2020.

[38] Byungmoon Kim, Yingjie Liu, Ignacio Llamas, Xiangmin Jiao, and Jarek Rossignac. Simulation of bubbles in foam with the volume control method. *ACM Transactions on Graphics (TOG)*, 26(3):98–es, 2007.

[39] ByungMoon Kim, Yingjie Liu, Ignacio Llamas, and Jarek Rossignac. Flowfixer: Using bfecc for fluid simulation. In *NPH*, pages 51–56, 2005.

[40] Chenfanfu Jiang, Craig Schroeder, Andrew Selle, Joseph Teran, and Alexey Stomakhin. Tech report: the affine particle-in-cell method. 2025.